Spring Boot & Microservices Interview Mastery

The Senior Software Engineer Track
Complete Master Edition — Modules 1–15

Targeting interviews at Google · Amazon · Microsoft · Uber · Netflix · LinkedIn · VMware · Broadcom · Oracle · JPMorgan · Goldman Sachs · Walmart Global Tech · Adobe · Atlassian

For engineers moving from Node.js / MERN to Java & Spring Boot.
Each topic: why it's asked · internals · lifecycle · ASCII diagrams · production examples · trade-offs · traps · debugging · follow-ups · exercises

Generated July 03, 2026

Contents

Module 1 — Core Java for Spring Interviews

> For engineers coming from Node.js / MERN. Java is statically typed, compiled to bytecode, and runs on a managed runtime (the JVM) with real OS threads and a sophisticated garbage collector. Almost every Spring internal — bean proxies, `@Transactional`, thread pools, `CompletableFuture` — is built on the primitives in this module. Interviewers probe here to see whether you understand the *platform*, not just the syntax.

1.1 JVM Architecture, JDK vs JRE vs JVM

Why Interviewers Ask This

They want to know if you understand what actually runs your code, how "write once run anywhere" works, and where to look when a service OOMs or stalls in production. Coming from Node (single V8 process, event loop), you must show you grasp the multi-threaded, managed-memory JVM model.

Core Concept

- **JVM (Java Virtual Machine):** the abstract spec + a concrete process that loads bytecode (`.class`), verifies it, JIT-compiles hot paths to native code, manages memory, and runs threads. It is *platform-specific* (different binary per OS/arch) but runs *platform-independent* bytecode.
- **JRE (Java Runtime Environment):** JVM + core class libraries (`java.*`, `java.util.*`) needed to *run* apps. No compiler.
- **JDK (Java Development Kit):** JRE + developer tools (`javac`, `jar`, `javap`, `jstack`, `jmap`, `jcmd`, `jshell`). Needed to *build* apps.

Relationship: `JDK ⊃ JRE ⊃ JVM`.

Internal Working

Source `.java` → `javac` → bytecode `.class` → class loader loads into JVM → bytecode verifier checks safety → interpreter executes; the **JIT compiler** (C1 client + C2 server, tiered compilation) profiles and compiles "hot" methods to optimized native code. HotSpot also does inlining, escape analysis (stack allocation of non-escaping objects), and deoptimization when assumptions break.

Lifecycle / Execution Flow

```
.java --javac--> .class (bytecode)
                    |
                ClassLoader (load)
                    |
             Bytecode Verifier
                    |
        Interpreter  <----->  JIT (C1/C2)  --> native code cache
                    |
           Runtime Data Areas (Heap, Stacks, Metaspace, PC, Native)
                    |
             Execution Engine + GC threads
```

ASCII Diagram — JVM Runtime Structure

```
+--------------------------- JVM PROCESS ---------------------------+
|  Class Loader Subsystem: Bootstrap -> Platform -> Application      |
|--------------------------------------------------------------------|
|  RUNTIME DATA AREAS                                                |
|   +---------------+   Shared across threads                       |
|   |     HEAP      |  (Young: Eden+S0/S1, Old)                     |
|   +---------------+                                               |
|   |   METASPACE   |  (class metadata, native memory)              |
|   +---------------+                                               |
|   Per-thread:                                                     |
|   [JVM Stack][PC Register][Native Method Stack]  x N threads      |
|--------------------------------------------------------------------|
|  EXECUTION ENGINE: Interpreter | JIT (C1/C2) | GC | Threads       |
+--------------------------------------------------------------------+
```

Real Production Example

A Spring Boot service is packaged as a fat JAR and shipped in a container built `FROM eclipse-temurin:21-jre` (JRE only — smaller image, no compiler needed at runtime). You pass `-XX:MaxRAMPercentage=75` so the JVM sizes its heap relative to the container memory limit rather than the host.

Advantages

Portability, mature JIT (often near-native throughput), rich tooling, managed memory (no manual free), massive ecosystem.

Trade-offs

JVM warm-up latency (JIT needs profiling before peak performance), higher memory baseline than Node, GC pauses to reason about. Mitigations: AOT/CDS, GraalVM native image, `-XX:+UseAppCDS`.

Common Mistakes

Shipping a full JDK in prod images; ignoring container memory limits (pre-JDK 10 JVMs read host memory, not cgroup limits → OOMKilled); confusing JRE and JDK.

Performance Considerations

Tiered compilation reaches peak throughput after warm-up. Use `-Xss` for stack size, `-Xmx/-Xms` (or `MaxRAMPercentage`) for heap. CDS (Class Data Sharing) speeds startup.

Debugging Techniques

`java -XX:+PrintFlagsFinal -version`, `jcmd <pid> VM.flags`, `jinfo`, `jstack` for thread dumps, `jmap`/`jcmd GC.heap_dump` for heap dumps.

Common Interview Questions

- Difference between JDK, JRE, JVM?
- Is Java compiled or interpreted? *(Both: compiled to bytecode, then interpreted + JIT-compiled.)*
- What is the JIT and tiered compilation?

Follow-up Questions

- How does the JVM decide a method is “hot”? *(invocation + back-edge counters.)*
- What is deoptimization? What is escape analysis?

Hands-on Coding Exercise

Compile a class with `javac`, then run `javap -c` to inspect its bytecode and identify the `invokevirtual` / `invokedynamic` opcodes.

Best Practices

Use a JRE (or `jlink`-trimmed runtime) in prod images; always set memory flags relative to the container; standardize on an LTS (17 or 21).

1.2 Java Memory Model: Heap, Stack, Metaspace

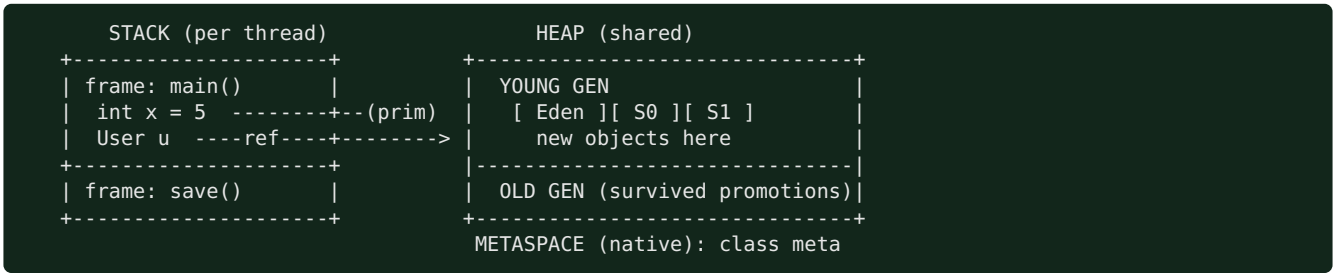
Why Interviewers Ask This

Memory questions reveal whether you can diagnose OOMs, leaks, and GC pressure — the #1 production Java incident.

Core Concept

- **Heap** — shared, GC-managed; all objects and arrays live here. Split into **Young Gen** (Eden + two Survivor spaces S0/S1) and **Old/Tenured Gen**.
- **Stack** — per-thread; stores frames (local variables, operand stack, return address). Primitives and *references* live here; the objects they point to live on the heap. `StackOverflowError` when too deep.
- **Metaspace** — native (off-heap) memory holding class metadata. Replaced PermGen in Java 8. Grows dynamically; cap with `-XX:MaxMetaspaceSize`.
- **PC register** and **native method stack** — per thread.
- **The JMM** (JSR-133) also defines *visibility/ordering* rules: `happens-before`, `volatile`, `synchronized`, `final` semantics.

Internal Working & ASCII Diagram



Objects are allocated in Eden. A **minor GC** copies live objects Eden→Survivor, aging them; after N survivals they are **promoted** to Old Gen. A **major/full GC** collects Old Gen (more expensive).

Real Production Example

`java.lang.OutOfMemoryError: Java heap space` in a report exporter that loads a whole result set into a `List`. Fix: stream results (JPA `Stream`, pagination) so objects stay short-lived in Young Gen and die at minor GC.

Trade-offs / Common Mistakes

- Confusing the reference (stack) with the object (heap).
- Thinking Metaspace is on the heap — it isn't; a classloader leak (e.g. repeated hot redeploys, dynamic proxies) causes `OutOfMemoryError: Metaspace`.
- Huge thread counts × large `-Xss` → native OOM.

Performance / Debugging

`jmap -histo:live <pid>`, heap dump + Eclipse MAT to find the dominator tree and retained size; `-XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/dumps`.

Interview Q / Follow-ups

- Where do primitives vs objects live? *(local primitives/refs on stack; objects on heap.)*
- What replaced PermGen and why? *(Metaspace; auto-grows in native memory.)*

- Explain `happens-before`. What does `volatile` guarantee? *(visibility + ordering, not atomicity of compound ops.)*

Hands-on Exercise

Write a program that grows an unbounded `static List<byte[]>` and observe `OutOfMemoryError: Java heap space`; capture and open the heap dump.

Best Practices

Prefer short-lived objects; avoid static collections that never release; set `-XX:MaxMetaspaceSize`; always enable heap-dump-on-OOM in prod.

1.3 Garbage Collection: G1, ZGC, Shenandoah

Why Interviewers Ask This

GC tuning separates senior engineers. They want to know you can choose a collector by latency vs throughput requirements and read GC logs.

Core Concept

GC reclaims unreachable objects (reachability from **GC roots**: stack refs, statics, JNI). Modern collectors are **generational** and mostly concurrent.

Collector	Best for	Pause behavior	Notes
Serial	tiny heaps, single core	stop-the-world	`-XX:+UseSerialGC`
Parallel	batch/throughput	STW, multi-thread	max throughput
G1 (default 9+)	balanced, large heaps	~soft pause target	region-based
ZGC	very large heaps, low latency	<1ms, concurrent	colored pointers
Shenandoah	low latency (Red Hat)	<10ms	concurrent compaction

Internal Working

- **G1** divides the heap into equal **regions**; collects regions with most garbage first (“garbage first”). Uses SATB (snapshot-at-the-beginning) marking and honors a pause target (`-XX:MaxGCPauseMillis=200`).
- **ZGC / Shenandoah** do concurrent marking *and* concurrent relocation using load/read barriers (ZGC uses colored pointers + load barrier), so pauses stay sub-millisecond regardless of heap size (multi-TB).

ASCII Diagram — Generational Copying

```
Eden fills ---> Minor GC ---> live copied to Survivor ---> age++
  |                                                          |
  +----- dead objects reclaimed                    age >= threshold
                                                             |
                                                        promote to OLD
OLD fills ---> concurrent mark (G1/ZGC) ---> reclaim/compact
```

Real Production Example

A trading API with a strict p99 latency SLA switched from G1 to **ZGC** (`-XX:+UseZGC`) on a 64 GB heap; GC pauses dropped from ~120 ms to <1 ms, eliminating latency spikes, at a small throughput cost.

Advantages / Trade-offs

G1: good default, predictable. ZGC/Shenandoah: tiny pauses but slightly lower throughput and higher CPU/barrier overhead. Parallel: highest throughput but long STW pauses.

Common Mistakes

Blindly setting `-XX:MaxGCPauseMillis` too low (G1 shrinks young gen → frequent GC); ignoring allocation rate (real cause of GC pressure); calling `System.gc()`.

Performance / Debugging

Enable unified GC logs: `-Xlog:gc*:file=gc.log:tags,uptime,level`. Analyze with GCEasy/GCViewer. Watch allocation rate, promotion rate, pause times.

Interview Q / Follow-ups

- Difference between minor, major, full GC?
- Why is G1 the default? When choose ZGC?
- Can GC cause a memory leak? *(Yes — logical leaks: reachable-but-unused objects, e.g. static caches, unclosed listeners.)*
- What are GC roots?

Hands-on Exercise

Run the same allocation-heavy program with `-XX:+UseParallelGC`, `-XX:+UseG1GC`, and `-XX:+UseZGC`; compare pause times in the GC logs.

Best Practices

Start with G1; move to ZGC only for latency-critical, large-heap services; tune allocation rate before collector flags; always keep GC logs in prod.

1.4 Class Loading & Reflection

Core Concept

Class loaders load classes lazily, following the **parent-delegation model**: **Bootstrap** (core `java.*`) → **Platform/Extension** → **Application/System** (your classpath). A child delegates to its parent first, preventing core classes from being overridden.

ASCII Diagram

```
Bootstrap (rt/core)  <-- delegate
     ^
Platform loader      <-- delegate
     ^
Application loader (classpath / your JAR)
     ^
Custom loaders (Spring Boot's LaunchedURLClassLoader, web app loaders)
```

Spring Boot fat JARs use a **nested-jar** classloader that reads `BOOT-INF/lib`.

Reflection

Inspect/modify classes at runtime: `Class.forName`, `getDeclaredMethods`, `setAccessible(true)`. Spring uses reflection heavily for DI, to instantiate beans, inject fields, and read annotations.

Trade-offs / Mistakes / Debugging

Reflection is slower and breaks encapsulation; on JDK 17+ deep reflection into JDK internals needs `--add-opens`. `ClassNotFoundException` (not on classpath at runtime) vs `NoClassDefFoundError` (present at compile, missing/failed init at runtime) vs `ClassCastException`. Classloader leaks cause Metaspace OOM.

Interview Q / Follow-ups

- Explain parent delegation and why it exists.

- `ClassNotFoundException` vs `NoClassDefFoundError`?
- How does Spring create beans it never sees the concrete type of? *(reflection + proxies.)*

Hands-on Exercise

Use reflection to instantiate a class by name and invoke a private method via `setAccessible(true)`.

Best Practices

Prefer `MethodHandles`/`VarHandle` over raw reflection for hot paths; cache `Method`/`Field` lookups; avoid reflection in tight loops.

1.5 Generics, Collections, Comparable vs Comparator

Generics

Compile-time type safety with **type erasure** (types removed at runtime → no `new T[]`, no `instanceof T`). Wildcards: `? extends T` (producer, read), `? super T` (consumer, write) — **PECS: Producer Extends, Consumer Super**.

Collections Cheat Sheet

Interface	Impl	Ordering	Notes / Big-O
List	`ArrayList`	insertion	O(1) get, O(n) mid-insert
List	`LinkedList`	insertion	O(1) add ends, O(n) get
Set	`HashSet`	none	O(1) add/contains
Set	`LinkedHashSet`	insertion	predictable iteration
Set	`TreeSet`	sorted	O(log n), needs Comparable/Comparator
Map	`HashMap`	none	O(1) avg; treeifies buckets ≥8
Map	`LinkedHashMap`	insertion/access	LRU cache base
Map	`TreeMap`	sorted keys	O(log n)
Map	`ConcurrentHashMap`	none	lock-striped, thread-safe
Queue	`ArrayDeque`	FIFO/LIFO	fast stack/queue
Queue	`PriorityQueue`	heap order	O(log n) offer/poll

HashMap internals: array of buckets; key `hashCode()` spread → index; bucket holds a linked list, converted to a **red-black tree** when ≥8 entries (and table ≥64) to bound worst case at O(log n). Load factor 0.75 triggers resize (doubling).

Comparable vs Comparator

- `Comparable<T>` — natural ordering, `compareTo`, one per class (`implements`).
- `Comparator<T>` — external/custom ordering, multiple allowed; `comparing`, `thenComparing`, `reversed`.

ASCII — HashMap Bucket

```
index = (n-1) & spread(hash(key))
bucket[5] -> (k1,v1) -> (k2,v2) -> ... (list)  --treeify(>=8)-->  RB-tree
```

Interview Q / Follow-ups

- How does `HashMap` work? What happens on collision / resize?
- Why override `equals` and `hashCode` together?
- `HashMap` vs `ConcurrentHashMap` vs `Collections.synchronizedMap`?
- `Comparable` vs `Comparator`; how to sort by multiple fields?

Hands-on Exercise

Sort a `List<Employee>` by department asc then salary desc using `Comparator.comparing(Employee::dept).thenComparing(Employee::salary, reverseOrder())`.

Common Mistakes / Best Practices

Mutable keys in a `HashMap`; overriding `equals` without `hashCode`; using `Vector`/`Hashtable` (legacy). Prefer `ConcurrentHashMap` over synchronized maps; program to interfaces (`List`, `Map`).

1.6 Streams, Optional, Functional Interfaces & Lambdas

Core Concept

- **Functional interface:** exactly one abstract method (`@FunctionalInterface`). Key ones: `Function<T,R>`, `Supplier<T>`, `Consumer<T>`, `Predicate<T>`, `BiFunction`, `Runnable`, `Callable`.
- **Lambda:** anonymous implementation of a functional interface; compiled via `invokedynamic` (not an anonymous class) → cheaper.
- **Streams:** declarative pipelines — a **source** → **intermediate** (lazy: `map`, `filter`, `sorted`) → **terminal** (eager: `collect`, `reduce`, `forEach`). Support `parallel()` (uses common ForkJoinPool).
- **Optional:** container to express "maybe absent" and avoid `NullPointerException`; use `map`, `filter`, `orElseGet`, `orElseThrow` — **never** call `get()` blindly.

ASCII — Stream Pipeline

```
source -> filter -> map -> sorted -> collect
 (lazy intermediate ops build a pipeline; nothing runs)
                            └── terminal op triggers execution
```

Real Production Example

```
Map<String, List<Order>> byUser = orders.stream()
    .filter(o -> o.getTotal().compareTo(BigDecimal.TEN) > 0)
    .collect(Collectors.groupingBy(Order::getUserId));
```

Trade-offs / Common Mistakes

Streams are less debuggable than loops and can be slower for trivial cases; `parallelStream()` on small data or with shared mutable state hurts (and shares the common pool with the whole app). `Optional` as a *field* or method parameter is an anti-pattern — use it as a return type.

Interview Q / Follow-ups

- `map` vs `flatMap`? Intermediate vs terminal ops? Why lazy?
- When is `parallelStream()` a bad idea?
- Why `orElseGet` over `orElse` for expensive defaults? *(`orElse` always evaluates its argument.)*

Hands-on Exercise

Given `List<Transaction>`, compute total amount per currency and the top-3 highest transactions using streams.

Best Practices

Keep pipelines side-effect-free; return `Optional` (not null) for "maybe"; avoid parallel streams unless data is large, CPU-bound, and stateless.

1.7 Exception Handling

Core Concept

- **Checked** (`extends Exception`) — must be declared/handled; for recoverable conditions (`IOException`).
- **Unchecked** (`extends RuntimeException`) — programming errors (`NullPointerException`, `IllegalArgumentException`); not required to declare.
- **Error** (`OutOfMemoryError`) — don't catch.
- `try-with-resources` auto-closes `AutoCloseable`; `finally` for cleanup; multi-catch `catch (A | B e)`.

Common Mistakes / Best Practices

Swallowing exceptions (`catch (Exception e) {}`); catching `Throwable`; losing the cause (always chain: `throw new ServiceException("msg", e)`); using exceptions for control flow. In Spring, translate to a global `@ControllerAdvice`.

Interview Q

Checked vs unchecked; when to create custom exceptions; why is `throws Exception` bad on public APIs?

1.8 Concurrency: Threads, ExecutorService, Thread Pools, CompletableFuture

Why Interviewers Ask This

This is the biggest gap for Node developers. Java uses real OS threads and shared mutable memory, so you must reason about visibility, atomicity, and pools.

Core Concept

- **Thread** — unit of execution; `Runnable`/`Callable`. Creating threads per task is expensive → use pools.
- **ExecutorService / Thread Pool** — reuse a fixed set of worker threads pulling from a task queue. Create via `ThreadPoolExecutor` (avoid `Executors` factories with unbounded queues in prod).
- **Key params:** `corePoolSize`, `maxPoolSize`, `keepAlive`, `workQueue`, `RejectedExecutionHandler`.
- **CompletableFuture** — composable async: `supplyAsync`, `thenApply`, `thenCompose` (flatMap), `thenCombine`, `exceptionally`, `allOf`. This is the closest thing to Node's Promises, but backed by a thread pool.
- **Synchronization:** `synchronized`, `ReentrantLock`, `volatile`, atomics (`AtomicInteger`, `LongAdder`), `ConcurrentHashMap`.
- **Virtual threads (Java 21, Project Loom):** cheap user-mode threads for high I/O concurrency — millions per JVM.

ASCII — Thread Pool

```
submit(task) -> [ work queue ]  ->  worker1  worker2 ... workerN
                      |                 (core..max threads)
                queue full & max reached -> RejectedExecutionHandler
```

CompletableFuture flow

```
supplyAsync(fetchUser) --thenCompose--> fetchOrders(user)
       \--thenCombine(fetchProfile)--> merge --exceptionally--> fallback
```

Real Production Example

Aggregating three downstream calls in parallel and merging with a timeout:

```
CompletableFuture<User>    u = supplyAsync(() -> userClient.get(id), pool);
CompletableFuture<Profile> p = supplyAsync(() -> profileClient.get(id), pool);
return u.thenCombine(p, ProfileView::of)
        .orTimeout(500, MILLISECONDS)
        .exceptionally(ex -> ProfileView.fallback(id));
```

Trade-offs / Common Mistakes

- Using the default `ForkJoinPool.commonPool` for blocking I/O (starves it) — pass an explicit pool to `*Async`.
- Unbounded queues (`Executors.newFixedThreadPool`) hide backpressure → OOM.
- Data races from unsynchronized shared mutable state; deadlocks from lock ordering; `.get()` blocking defeats async.
- Thread pool sizing: CPU-bound ≈ #cores; I/O-bound larger (Little's Law).

Performance / Debugging

Thread dump `jstack <pid>` (find BLOCKED/WAITING, deadlocks); look for pool saturation, `RejectedExecutionException`, threads stuck on locks. Async Profiler / JFR for CPU + lock contention.

Interview Q / Follow-ups

- Runnable vs Callable? `Future` vs `CompletableFuture`?
- Explain `ThreadPoolExecutor` params and rejection policies.
- How do you size a thread pool? CPU-bound vs I/O-bound.
- `synchronized` vs `ReentrantLock`? What does `volatile` guarantee?
- What are virtual threads and when do they help? *(massive blocking I/O concurrency; not CPU-bound work.)*
- How to prevent/detect deadlock? *(consistent lock ordering, tryLock w/ timeout, thread dump.)*

Hands-on Exercise

Build a `ThreadPoolExecutor(4, 8, 60s, ArrayBlockingQueue(100), CallerRunsPolicy)`, submit 1000 tasks, and observe backpressure via `CallerRunsPolicy`.

Best Practices

Always use bounded queues + explicit pools; name threads (`ThreadFactory`) for debuggability; prefer `CompletableFuture`/structured concurrency over raw threads; never share mutable state without synchronization.

Module 1 — One-Page Cheat Sheet

Topic	Must-know
JDK/JRE/JVM	JDK⊃JRE⊃JVM; JIT tiered (C1/C2); bytecode is portable
Memory	Heap (Young Eden+S0/S1, Old), Stack (per-thread frames), Metaspace (native)
GC	G1 default; ZGC/Shenandoah = sub-ms pauses; roots = stack/statics/JNI
Collections	HashMap treeifies at 8; ConcurrentHashMap for concurrency; PECS
equals/hashCode	Override together; immutable keys
Streams	Lazy intermediate + eager terminal; avoid parallel by default
Optional	Return type only; `orElseGet` for expensive defaults
Exceptions	Checked=recoverable; chain cause; never swallow
Concurrency	Bounded pools + explicit executor; CompletableFuture composes; volatile≠atomic
Virtual threads	Java 21; great for blocking I/O, not CPU-bound

Module 1 — Top Interview Questions

1. Explain JVM memory areas and where objects vs references live.
2. How does `HashMap` work internally (collision, treeify, resize)?
3. Compare G1, ZGC, Shenandoah — when use each?
4. `Comparable` vs `Comparator`; sort by multiple fields.
5. `map` vs `flatMap`; why are streams lazy?
6. Explain `ThreadPoolExecutor` parameters and rejection policies.
7. `Future` vs `CompletableFuture`; compose parallel calls with timeout/fallback.
8. What does `volatile` guarantee; `synchronized` vs `ReentrantLock`.
9. Checked vs unchecked exceptions; exception chaining.
10. Virtual threads — what problem do they solve?

Module 1 — Common Mistakes

- Overriding `equals` without `hashCode`.
- Blocking I/O on the common ForkJoinPool.
- Unbounded thread-pool queues → OOM.
- Calling `Optional.get()` without checking; `Optional` as a field.
- Swallowing exceptions / losing the root cause.

Module 1 — Mock Interview (5 rapid-fire)

1. *"Our service is OOMing with `Java heap space`. Walk me through diagnosis."* → enable heap-dump-on-OOM, take dump, open in MAT, find dominators/retained size, look for unbounded static collections or a loaded-everything query; fix by streaming/paginating.
2. *"p99 latency spikes every few minutes."* → likely GC pauses; check GC logs, allocation rate; consider ZGC or reduce allocation.
3. *"Two threads hang forever."* → deadlock; take `jstack`, look for "Found one Java-level deadlock"; enforce lock ordering.
4. *"Why not just create a thread per request like Node handles requests?"* → OS threads are heavy (~1MB stack); use pools or virtual threads.
5. *"Combine three REST calls, return in 300ms or fallback."* → three `supplyAsync` on a bounded pool + `allOf` / `thenCombine` + `orTimeout` + `exceptionally`.

Next → Module 2: Spring Core (IoC, DI, bean lifecycle & the container).

Module 2 — Spring Core (IoC, DI, Bean Lifecycle & the Container)

> This is the highest-priority module. Everything in Spring is a bean living in an **ApplicationContext**. Interviewers want to hear *how the container works internally* — not just "`@Autowired` injects dependencies."

2.1 Inversion of Control (IoC) & Dependency Injection (DI)

Why Interviewers Ask This

It's the foundational idea. Coming from Node (where you `require()` and `new` things yourself), you must articulate why handing object creation to a container improves testability, decoupling, and lifecycle management.

Core Concept

- **IoC** — you don't create/wire your dependencies; the *container* does. Control of object graph construction is *inverted* to the framework.
- **DI** — the mechanism: dependencies are *injected* (constructor, setter, field) rather than looked up. Program to interfaces; the container supplies impls.

Internal Working

The container reads bean definitions (from annotations/`@Configuration`/XML), builds a `BeanDefinition` registry, resolves dependency graph order, instantiates singletons eagerly at startup, and injects collaborators. Backed by `DefaultListableBeanFactory`.

ASCII — IoC Container

```
  @Configuration / @ComponentScan
            |
      BeanDefinitionReader ---> [ BeanDefinition registry ]
            |                              |
      BeanFactoryPostProcessors (modify definitions, e.g. @Value placeholders)
            |
      instantiate + inject (resolve graph) ---> singleton cache
            |
      BeanPostProcessors (wrap: AOP proxies, @Async, @Transactional)
            |
        ApplicationContext (ready)
```

Real Production Example

A `PaymentService` depends on a `PaymentGateway` interface. Prod wires `StripeGateway`; tests inject a `MockGateway`. No code change — just different beans/profiles. That swap-ability is the payoff of DI.

Advantages / Trade-offs

- Loose coupling, testability (inject mocks), centralized lifecycle, AOP hooks. − Startup magic can be hard to trace; indirection; slower cold start; runtime (not compile-time) wiring errors if misconfigured.

Common Mistakes

Doing `new` inside beans (defeats DI); calling `context.getBean()` manually (service locator anti-pattern); mixing business logic with wiring.

Interview Q / Follow-ups

- IoC vs DI — are they the same? *(IoC is the principle; DI is one implementation.)*

- Types of DI and which is best? *(constructor — see 2.5.)*
- `BeanFactory` vs `ApplicationContext`? *(ApplicationContext = BeanFactory + i18n, events, AOP, auto post-processors, eager singletons.)*

Hands-on Exercise

Define a `NotificationService` interface with `EmailImpl` and `SmsImpl`; inject the desired one via `@Qualifier`/`@Primary`.

Best Practices

Program to interfaces; constructor injection; never use the container as a service locator in business code.

2.2 Bean Lifecycle

Why Interviewers Ask This

Reveals depth: initialization order, `BeanPostProcessor`, and where proxies are applied (critical for understanding `@Transactional`/`@Async`).

Core Concept & Execution Flow

```
1. Instantiate (constructor)
2. Populate properties / inject dependencies
3. Aware callbacks: BeanNameAware, BeanFactoryAware, ApplicationContextAware
4. BeanPostProcessor.postProcessBeforeInitialization
5. @PostConstruct
6. InitializingBean.afterPropertiesSet()  /  @Bean(initMethod=...)
7. BeanPostProcessor.postProcessAfterInitialization   <-- AOP proxy created here
8. Bean is READY (in use)
   ... container shutdown ...
9. @PreDestroy
10. DisposableBean.destroy()  /  @Bean(destroyMethod=...)
```

Internal Working

`BeanPostProcessor` is the extension point: Spring's own `AbstractAutoProxyCreator` wraps beans in AOP proxies during *after* initialization. That's why a self-invocation (`this.method()`) bypasses `@Transactional`/`@Async` — it doesn't go through the proxy.

ASCII — Where the Proxy Wraps

```
  raw bean  --BPP.after-->  [ Proxy (Tx/Async advice) ]  --> injected everywhere
  caller -> proxy.method() -> advice (begin tx) -> target.method() -> commit
  BUT: target.this.other()  bypasses proxy => no advice!
```

Real Production Example

`@PostConstruct` warms a cache after DI completes; `@PreDestroy` gracefully drains a connection pool on shutdown.

Common Mistakes / Debugging

Relying on field-injected deps inside the constructor (not injected yet); heavy work in constructors; self-invocation breaking transactions; expecting `@PreDestroy` on prototype beans (Spring doesn't manage prototype destruction).

Interview Q / Follow-ups

- Full bean lifecycle order.
- `@PostConstruct` vs `InitializingBean` vs `@Bean(initMethod)`.
- Why does self-invocation break `@Transactional`? *(proxy bypass — see AOP.)*
- What is a `BeanPostProcessor` vs `BeanFactoryPostProcessor`? *(BFPP edits definitions before instantiation; BPP edits bean instances.)*

Hands-on Exercise

Implement a bean with `@PostConstruct`, `InitializingBean`, and a custom `BeanPostProcessor` logging each phase; observe the order.

Best Practices

Prefer `@PostConstruct` / `@PreDestroy`; keep constructors cheap; never rely on self-invocation for AOP behavior — call through an injected reference or split into another bean.

2.3 Bean Scopes

Scope	Meaning	Use
singleton (default)	one instance per container	stateless services
prototype	new instance per `getBean` /injection	stateful/short-lived
request	one per HTTP request	web only
session	one per HTTP session	web only
application	one per ServletContext	web only
websocket	one per WebSocket session	web only

Gotcha — singleton depends on prototype: the prototype is injected *once* at singleton creation, so you don't get a fresh instance per call. Fix with `@Lookup` method injection, `ObjectProvider<T>`, or a `Provider<T>`.

Interview Q

- Are Spring singletons the same as GoF singletons? *(No — one per container, not per JVM; and not `private` constructors.)*
- Are singleton beans thread-safe? *(Only if stateless — you must ensure it.)*
- How to inject a prototype into a singleton correctly?

2.4 Stereotype Annotations & Configuration

`@Component` is the generic stereotype; the others are specializations picked up by component scanning:

Annotation	Layer	Extra behavior
`@Component`	generic	base for the rest
`@Service`	business logic	semantic only
`@Repository`	persistence	**exception translation** (`PersistenceExceptionTranslationPostProcessor` → `DataAccessException`)
`@Controller`	web MVC	view-returning handlers
`@RestController`	web REST	`@Controller` + `@ResponseBody`
`@Configuration`	config	**CGLIB-proxied** so `@Bean` methods return singletons
`@Bean`	factory method	programmatic bean registration (3rd-party types)

`@Configuration` internal trick

`@Configuration` classes are enhanced by CGLIB so that inter-bean method calls (`beanA()` calling `beanB()`) return the *shared singleton*, not a new instance. With `@Configuration(proxyBeanMethods=false)` (Spring Boot's "lite" mode) that interception is skipped for speed — then each call makes a new object.

`@Bean` vs `@Component`

- `@Component` — Spring instantiates the class you own (component scan).
- `@Bean` — you write a factory method; use for third-party classes you can't annotate (e.g. `ObjectMapper`, `RestTemplate`, a `DataSource`).

Interview Q / Follow-ups

- `@Component` vs `@Service` vs `@Repository` — real differences? *(mostly semantic; `@Repository` adds exception translation.)*
- Why is `@Configuration` proxied? What is `proxyBeanMethods=false`?
- When `@Bean` over `@Component`?

2.5 Component Scan, Autowiring & Injection Types

Component Scan

`@ComponentScan(basePackages=...)` (implicit via `@SpringBootApplication` on the main package) discovers stereotyped classes and registers `BeanDefinition`s. Filters: `includeFilters` / `excludeFilters`.

Autowiring resolution order

1. **By type** (the default for a single candidate).
2. If multiple candidates: `@Primary` wins, else `@Qualifier("name")`, else match by field/param **name**, else `NoUniqueBeanDefinitionException`.
3. No candidate + `required=true` → `NoSuchBeanDefinitionException` (use `ObjectProvider` / `@Autowired(required=false)` / `Optional<T>` for optional).

Constructor vs Field vs Setter Injection

	Constructor	Field	Setter
Immutability	`final`		
Required deps enforced			
Testable without Spring	(just `new`)	(reflection)	partial
Circular deps	fails fast (good)	silently allowed	allowed
Recommended	**Yes**	No	optional deps

Since Spring 4.3, a single constructor needs no `@Autowired`. Field injection is discouraged (hidden deps, hard to test, can't be `final`).

ASCII — Autowiring Decision

```
need bean of type T
  |-- 1 candidate --------------> inject it
  |-- N candidates:
  |      @Primary present? ------> inject primary
  |      @Qualifier given? ------> inject named
  |      name matches param? ----> inject by name
  |      else -------------------> NoUniqueBeanDefinitionException
  |-- 0 candidates --------------> NoSuchBeanDefinitionException (unless optional)
```

Interview Q / Follow-ups

- Constructor vs field injection — why is constructor preferred?
- How does Spring resolve multiple beans of the same type?
- `@Primary` vs `@Qualifier`?
- What triggers `NoUniqueBeanDefinitionException`?

Hands-on Exercise

Create two `PaymentGateway` beans; make one `@Primary`, inject the other with `@Qualifier`; then switch to `ObjectProvider` to pick at runtime.

2.6 Circular Dependencies

Core Concept

`A → B → A`. Spring resolves **setter/field** singleton cycles using a **three-level cache** (early exposure of a raw, not-yet-initialized reference):

```
singletonObjects (fully ready)
earlySingletonObjects (raw, exposed early)
singletonFactories (factory to build early ref)
```

Flow: creating A → put A's factory in level-3 → inject B → B needs A → gets A's *early* reference from cache → B finishes → A finishes. Works because the reference exists before initialization completes.

Constructor injection cycles cannot be resolved (the object doesn't exist yet to expose) → `BeanCurrentlyInCreationException`. Spring Boot 2.6+ also **prohibits circular references by default** (must set `spring.main.allow-circular-references=true`).

ASCII

```
create A -> expose earlyRef(A) -> inject into A: need B
    -> create B -> inject earlyRef(A) into B -> B done
    -> back to A -> A done
(Constructor cycle: no earlyRef possible -> exception)
```

Real Production Example / Fix

A cycle usually signals a design smell. Fixes: **refactor** (extract a third component), break with `@Lazy` on one dependency (injects a proxy), or use setter injection / `ApplicationEventPublisher`.

Interview Q / Follow-ups

- How does Spring resolve circular deps? Why only setter/field, not constructor?
- Explain the three-level cache.
- How does `@Lazy` break a cycle? *(injects a lazy proxy; real bean resolved on first use.)*
- Why did Spring Boot disable circular refs by default?

2.7 Profiles, Environment & Property Sources

Core Concept

- **`Environment`** abstracts profiles + properties. Beans/config can be activated per **profile** (`@Profile("prod")`).
- **Property sources** are layered with a defined precedence; `@Value("${...}")` and `Environment.getProperty()` read them.

Spring Boot property precedence (high → low, abridged)

```
1. Command-line args (--server.port=9090)
2. OS environment variables / SPRING_APPLICATION_JSON
3. application-{profile}.properties/yml (profile-specific)
4. application.properties/yml (default)
5. @PropertySource, defaults
```

Env vars map: `SERVER_PORT` → `server.port` (relaxed binding).

Real Production Example

`application-prod.yml` points at the real DB and Kafka cluster; `-Dspring.profiles.active=prod` (or `SPRING_PROFILES_ACTIVE=prod`) selects it. `@Profile("!prod")` beans provide in-memory test doubles locally.

Interview Q / Follow-ups

- How do profiles work; how to activate them?
- Property source precedence — where does an env var rank vs `application.yml`?
- `@Value` vs `@ConfigurationProperties`? *(latter is type-safe, grouped, validated, relaxed-bound — preferred for many props.)*
- What is relaxed binding?

Hands-on Exercise

Bind a `@ConfigurationProperties(prefix="app.mail")` record; override one field via an environment variable and confirm precedence.

Best Practices

Never hardcode secrets; externalize per environment; prefer `@ConfigurationProperties` for groups; keep profile-specific overrides minimal.

Module 2 — One-Page Cheat Sheet

Concept	Key point
IoC/DI	Container owns object graph; inject via constructor
Context	`ApplicationContext` = `BeanFactory` + events/AOP/i18n/eager singletons
Lifecycle	instantiate→inject→aware→BPP.before→@PostConstruct→afterPropertiesSet→BPP.after(proxy)→ready→@PreDestroy→de
Proxy	Created in BPP-after; self-invocation bypasses AOP
Scopes	singleton default; prototype not destroyed by container; use ObjectProvider
Stereotypes	@Repository adds exception translation; @Configuration CGLIB-proxied
@Bean vs @Component	@Bean for 3rd-party types
Autowire	type→@Primary→@Qualifier→name
Injection	constructor (final, testable, fail-fast) preferred
Circular	3-level cache fixes setter/field, not constructor; disabled by default 2.6+
Profiles/ Props	layered precedence; cmd-line > env > profile yml > default yml

Module 2 — Top Interview Questions

1. Explain the full bean lifecycle and where AOP proxies are applied.

2. Why does self-invocation break `@Transactional`/`@Async`?
3. Constructor vs field injection — trade-offs.
4. How does Spring resolve circular dependencies (three-level cache)? Constructor limitation?
5. `@Component` vs `@Bean` vs `@Configuration`; why is `@Configuration` proxied?
6. `BeanFactory` vs `ApplicationContext`; `BeanPostProcessor` vs `BeanFactoryPostProcessor`.
7. Bean scopes; injecting prototype into singleton.
8. How does autowiring resolve ambiguity? `@Primary` vs `@Qualifier`.
9. Profiles and property-source precedence.
10. `@Value` vs `@ConfigurationProperties`.

Module 2 — Common Mistakes

- Field injection everywhere (untestable, hidden deps).
- Expecting `@Transactional` to work on self-invoked / private methods.
- Assuming singleton beans are automatically thread-safe.
- Injecting a prototype into a singleton and expecting freshness.
- Business code calling `getBean()` (service-locator anti-pattern).

Module 2 — Mock Interview

1. *"`@Transactional` on `save()` isn't rolling back when called from `process()` in the same class. Why?"* → self-invocation bypasses the proxy; move `save()` to another bean or call via injected self-proxy.
2. *"App won't start: `BeanCurrentlyInCreationException`."* → constructor circular dependency; refactor or `@Lazy`.
3. *"Two `DataSource` beans, injection fails."* → `NoUniqueBeanDefinitionException`; add `@Primary`/`@Qualifier`.
4. *"How do you supply different configs for dev/staging/prod without rebuilding?"* → profiles + externalized `application-{profile}.yml` + env vars.
5. *"Walk me through what happens between JVM start and the context being ready."* → read definitions → BFPPs → instantiate → inject → BPPs (proxies) → `@PostConstruct` → ready.

Next → Module 3: Spring Boot (auto-configuration, starters, actuator).

Module 3 — Spring Boot

> Spring Boot = Spring + opinionated auto-configuration + embedded server + production-ready features.
> Interviewers want to know *how the magic works* (auto-config, starters, the embedded container) so you can debug it.

3.1 Auto-Configuration

Why Interviewers Ask This

It's *the* Spring Boot differentiator. If you can explain how Boot configures a `DataSource` you never declared, you understand the framework, not just the demo.

Core Concept

Auto-configuration conditionally registers beans based on what's on the classpath, existing beans, and properties — so sensible defaults appear automatically, yet you can always override them.

Internal Working

- `@SpringBootApplication` = `@SpringBootConfiguration` + `@ComponentScan` + **`@EnableAutoConfiguration`**.
- `@EnableAutoConfiguration` imports `AutoConfigurationImportSelector`, which reads candidate config classes from `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` (pre-2.7: `spring.factories`).
- Each candidate is guarded by **`@Conditional`** annotations:
- `@ConditionalOnClass` / `@ConditionalOnMissingClass`
- `@ConditionalOnBean` / `@ConditionalOnMissingBean` (your bean wins!)
- `@ConditionalOnProperty`
- `@ConditionalOnWebApplication`
- Ordering via `@AutoConfigureBefore/After/Order`.

ASCII — Auto-config Flow

```
@SpringBootApplication
  -> @EnableAutoConfiguration
       -> AutoConfigurationImportSelector
            -> read AutoConfiguration.imports (100s of candidates)
                 -> for each: evaluate @Conditional*
                      pass? register beans (DataSource, DispatcherServlet, ...)
                      user already defined bean? @ConditionalOnMissingBean -> skip
```

Real Production Example

Add `spring-boot-starter-data-jpa` + an H2 dependency → Boot auto-creates a `DataSource`, `EntityManagerFactory`, `JpaTransactionManager`, and configures Hibernate — zero XML. Define your own `DataSource` bean and Boot backs off (`@ConditionalOnMissingBean`).

Advantages / Trade-offs

- Massive boilerplate reduction, consistent defaults, override-friendly. − "Magic" can be opaque; classpath changes silently alter behavior; startup does a lot of conditional evaluation.

Common Mistakes / Debugging

Assuming a bean is missing when auto-config backed off (or vice versa). Use the **condition evaluation report**: run with `--debug` (or `/actuator/conditions`) to see *why* each auto-config matched or not. Exclude with `@SpringBootApplication(exclude = DataSourceAutoConfiguration.class)`.

Interview Q / Follow-ups

- How does auto-configuration work end to end?
- What is `@ConditionalOnMissingBean` and why does it matter for overriding?
- Where are auto-config classes registered? *(`AutoConfiguration.imports`.)*
- How do you debug why a bean was/wasn't created? *(`--debug` conditions report.)*
- How do you write your own starter/auto-config?

Hands-on Exercise

Write a custom auto-configuration class guarded by `@ConditionalOnProperty` and `@ConditionalOnMissingBean`, register it in `AutoConfiguration.imports`, and verify it via the conditions report.

Best Practices

Prefer properties over disabling auto-config; use `@ConditionalOnMissingBean` in your own starters; keep the classpath intentional.

3.2 Starter Dependencies

Core Concept

Starters are curated, versioned dependency bundles (BOM-managed) — e.g. `spring-boot-starter-web` pulls Spring MVC, Jackson, validation, embedded Tomcat. The **parent BOM** (`spring-boot-dependencies`) fixes compatible versions so you don't manage them individually.

Interview Q

- What does a starter actually contain? *(transitive deps + version alignment, no code.)*
- How does Boot manage versions? *(dependency management BOM.)*
- Common starters: `-web`, `-data-jpa`, `-security`, `-actuator`, `-validation`, `-webflux`, `-test`.

3.3 Embedded Server (Tomcat)

Core Concept

Boot embeds the servlet container (default **Tomcat**; alternatives Jetty, Undertow) *inside* the fat JAR. `java -jar app.jar` starts an HTTP server — no external app server / WAR deployment.

Internal Working & Lifecycle

`SpringApplication.run()` → creates `ServletWebServerApplicationContext` → `ServletWebServerFactory` (Tomcat) starts the connector → registers the `DispatcherServlet` → binds the port. Graceful shutdown drains in-flight requests (`server.shutdown=graceful`).

ASCII

```
java -jar app.jar
  -> SpringApplication.run()
      -> create ApplicationContext (servlet-aware)
      -> TomcatServletWebServerFactory.getWebServer().start()
      -> register DispatcherServlet, filters
      -> listen on :8080
```

Trade-offs / Tuning

Tune `server.tomcat.threads.max` (default 200), `accept-count`, `max-connections`, `connection-timeout`. Undertow is lightweight; WebFlux/Netty for reactive. Thread-per-request means blocking calls consume worker threads.

Interview Q / Follow-ups

- Embedded vs external server — pros/cons? *(self-contained deploys, versioned per app vs shared infra.)*
- How to switch to Undertow/Jetty? *(exclude Tomcat starter, add the other.)*
- What is the default Tomcat thread pool size and how does it affect throughput?

3.4 Configuration Properties, Profiles & Logging

`@ConfigurationProperties`

Type-safe binding of a property group to a POJO/record; supports validation (`@Validated` + JSR-380), nested objects, lists, `Duration`/`DataSize`, and relaxed binding (`app.max-size` ↔ `APP_MAXSIZE`).

```
@ConfigurationProperties(prefix = "app.rate-limit")
public record RateLimitProps(int perMinute, Duration window) {}
```

Profiles

`application-{profile}.yml` + `spring.profiles.active`. Profile groups (`spring.profiles.group.prod=db,cache`). `@Profile` on beans.

Logging

Boot uses **SLF4J + Logback** by default. Configure levels via `logging.level.com.acme=DEBUG`; use `logback-spring.xml` for appenders/JSON encoding. Structured (JSON) logs for centralized logging; MDC for correlation IDs.

Interview Q

- `@Value` vs `@ConfigurationProperties` (type safety, grouping, validation).
- How to log JSON with a trace/correlation id? *(logback JSON encoder + MDC.)*

3.5 Validation

Core Concept

Bean Validation (JSR-380 / Jakarta Validation, Hibernate Validator impl). Annotate DTO fields (`@NotNull`, `@NotBlank`, `@Email`, `@Size`, `@Min`, `@Pattern`); trigger with `@Valid`/`@Validated` on controller params. Group validation and custom `ConstraintValidator` for domain rules.

```
public record CreateUser(@NotBlank String name,
                         @Email String email,
                         @Min(18) int age) {}

@PostMapping("/users")
User create(@Valid @RequestBody CreateUser dto) { ... }
```

Failures throw `MethodArgumentNotValidException` → handle globally (3.6).

Common Mistakes

Forgetting `@Valid`; validating entities instead of DTOs; `@Validated` (class level, needed for method-parameter validation on `@Service`) vs `@Valid`.

3.6 Exception Handling

Core Concept

Centralize error handling with **`@RestControllerAdvice`** + `@ExceptionHandler`, returning a consistent error body (ideally **RFC 7807 `ProblemDetail`**, built-in since Spring 6).

```
@RestControllerAdvice
class ApiExceptionHandler {
  @ExceptionHandler(MethodArgumentNotValidException.class)
  ProblemDetail onValidation(MethodArgumentNotValidException ex) {
    var pd = ProblemDetail.forStatus(HttpStatus.BAD_REQUEST);
    pd.setTitle("Validation failed");
    pd.setProperty("errors", ex.getBindingResult().getFieldErrors()
        .stream().collect(toMap(FieldError::getField, FieldError::getDefaultMessage)));
    return pd;
  }
  @ExceptionHandler(EntityNotFoundException.class)
  ProblemDetail onNotFound(EntityNotFoundException ex) {
    return ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND, ex.getMessage());
  }
}
```

Interview Q / Follow-ups

- `@ControllerAdvice` vs `@RestControllerAdvice`? *(latter adds `@ResponseBody`.)*
- How to return consistent error responses? *(ProblemDetail.)*
- Ordering multiple advices? `@Order`. Handling `ResponseStatusException`?

3.7 REST API Best Practices

- **Nouns + plurals**: `/orders/{id}/items`; no verbs in paths.
- **HTTP methods**: GET (safe/idempotent), POST (create), PUT (full replace, idempotent), PATCH (partial), DELETE (idempotent).
- **Status codes**: 200/201/204, 400/401/403/404/409/422, 429, 500/503.
- **Versioning**: URI (`/v1`), header, or media-type.
- **Pagination/filtering/sorting**; **HATEOAS** where useful.
- **Idempotency keys** for POST retries; **consistent error schema**; **DTOs** (never expose entities).

3.8 Filters vs Interceptors

	Filter (Servlet)	Interceptor (Spring MVC)
Layer	Servlet container, before DispatcherServlet	Inside MVC, around handler
API	`jakarta.servlet.Filter`	`HandlerInterceptor`
Access	raw request/response	handler, ModelAndView
Use	auth, CORS, gzip, logging, correlation id	auth on handlers, timing, MDC
Order	`@Order` / `FilterRegistrationBean`	registration order

ASCII — Where they sit

```
Client -> [ Servlet Filters ] -> DispatcherServlet -> [ Interceptors.preHandle ]
       -> Controller -> [ Interceptors.postHandle ] -> view/body
       -> [ Interceptors.afterCompletion ] -> [ Filters (unwind) ] -> Client
```

Interview Q

Filter vs interceptor vs AOP; where to set a correlation id (filter, early); `OncePerRequestFilter` and why.

3.9 Actuator

Core Concept

Production-ready endpoints under `/actuator`: `/health` (liveness/readiness groups), `/info`, `/metrics`, `/prometheus`, `/env`, `/loggers` (change log levels at runtime!), `/threaddump`, `/heapdump`, `/httpexchanges`, `/mappings`, `/conditions`.

Best Practices / Security

Expose only what you need (`management.endpoints.web.exposure.include=health,info,prometheus`); secure actuator (separate port/auth); use health groups for K8s liveness vs readiness probes.

Interview Q

- Difference between liveness and readiness probes and how actuator maps to them.
- How to change log level without redeploy? *(`/actuator/loggers` POST.)*

Module 3 — One-Page Cheat Sheet

Topic	Key point
@SpringBootApplication	=Config + ComponentScan + EnableAutoConfiguration
Auto-config	conditional beans from `AutoConfiguration.imports`; `@ConditionalOnMissingBean` lets you override
Debug config	`--debug` / `/actuator/conditions` report
Starters	curated deps + BOM version alignment
Embedded Tomcat	fat JAR, `java -jar`; ~200 worker threads
@ConfigurationProperties	type-safe, validated, relaxed binding
Validation	`@Valid` DTOs → `MethodArgumentNotValidException`
Errors	`@RestControllerAdvice` + `ProblemDetail` (RFC 7807)
Filter vs Interceptor	filter=servlet layer; interceptor=MVC layer
Actuator	health/metrics/loggers/prometheus; secure it

Module 3 — Top Interview Questions

1. Explain auto-configuration internals (`@Conditional`, imports file, back-off).
2. What does `@SpringBootApplication` combine?
3. How to override an auto-configured bean?
4. Embedded vs external server; tune the Tomcat thread pool.
5. `@Value` vs `@ConfigurationProperties`.
6. Global exception handling & consistent error responses.
7. Filter vs interceptor — when to use each.
8. Actuator: liveness vs readiness; runtime log level change.
9. How would you build a custom starter?
10. REST best practices (status codes, idempotency, versioning).

Module 3 — Common Mistakes

- Fighting auto-config instead of using properties/back-off.
- Exposing all actuator endpoints publicly.
- Forgetting `@Valid`; exposing entities instead of DTOs.
- Blocking calls saturating the fixed Tomcat thread pool.
- Not using health groups for K8s probes.

Module 3 — Mock Interview

1. *"Boot created a `DataSource` I didn't declare — how, and how do I override it?"* → auto-config via `@ConditionalOnClass` + `@ConditionalOnMissingBean`; declare your own bean and it backs off.
2. *"How do I know why bean X wasn't created?"* → conditions report (`--debug`).
3. *"Requests queue under load though CPU is idle."* → thread-per-request Tomcat pool exhausted by blocking downstream calls; raise pool / add timeouts / go reactive.
4. *"Standardize error responses across 40 endpoints."* → `@RestControllerAdvice` + `ProblemDetail`.

5. *“Change log level in prod without redeploy.”* → `POST /actuator/loggers/com.acme` `{"configuredLevel":"DEBUG"}`.

Next → Module 4: Spring MVC (DispatcherServlet & the request lifecycle).

Module 4 — Spring MVC

> Spring MVC is the servlet-based web framework. The single most-asked MVC question is: **“Walk me through the complete request lifecycle through the DispatcherServlet.”** Master that flow and you cover most of this module.

4.1 DispatcherServlet & the Complete Request Lifecycle

Why Interviewers Ask This

It proves you understand the front-controller pattern and every extension point (handler mapping, argument resolvers, message converters, exception resolvers).

Core Concept

`DispatcherServlet` is the **front controller** — a single servlet that receives every request and orchestrates specialized components to produce a response.

Internal Working — the components

- **HandlerMapping** — maps URL+method → handler (`RequestMappingHandlerMapping`).
- **HandlerAdapter** — invokes the handler (`RequestMappingHandlerAdapter`).
- **HandlerMethodArgumentResolver** — binds params (`@RequestBody`, `@PathVariable`, `@RequestParam`, `@RequestHeader`, `Pageable`).
- **HttpMessageConverter** — serialize/deserialize body (Jackson for JSON).
- **HandlerInterceptor** — pre/post/afterCompletion hooks.
- **ViewResolver** — for view-based responses (Thymeleaf/JSP); skipped for `@ResponseBody`/REST.
- **HandlerExceptionResolver** — maps exceptions to responses (`@ExceptionHandler`, `ResponseStatusException`).

Lifecycle / Execution Flow

```
1. Request hits Servlet filters, then DispatcherServlet.doDispatch()
2. getHandler(): HandlerMapping -> HandlerExecutionChain (handler + interceptors)
3. interceptor.preHandle()  (short-circuit if false)
4. HandlerAdapter: resolve arguments (ArgumentResolvers + MessageConverters)
5. Invoke controller method -> return value
6. Handle return: @ResponseBody -> MessageConverter writes body
                  else -> ViewResolver -> render view
7. interceptor.postHandle()
8. render / write response
9. interceptor.afterCompletion()
   (any exception -> HandlerExceptionResolver -> error response)
```

ASCII — Full Flow

```
Client
  |  HTTP request
  ▼
[Servlet Filters]  (CORS, security, correlation-id)
  ▼
DispatcherServlet ──► HandlerMapping ──► handler + interceptors
  |
  ├─► interceptor.preHandle()
  ├─► HandlerAdapter ─► ArgumentResolvers ─► MessageConverter (JSON->obj)
  |        ▼
  |    @RestController method
  |        ▼ returns object / view name
  ├─► MessageConverter (obj->JSON)  OR  ViewResolver->View
  ├─► interceptor.postHandle()
  └─► interceptor.afterCompletion()
  ▼
(exception anywhere) ─► HandlerExceptionResolver ─► @ExceptionHandler
  ▼
Response ─► [Filters unwind] ─► Client
```

Real Production Example

`POST /orders` with JSON: filters run → mapping finds `OrderController.create` → `@RequestBody` triggers Jackson to deserialize → `@Valid` validates → method runs → returns `Order` → Jackson serializes to JSON with `201 Created`.

Advantages / Trade-offs

Clean separation, huge extensibility. Thread-per-request (blocking) — for high I/O concurrency consider WebFlux. Lots of moving parts to learn.

Common Mistakes / Debugging

Missing `@ResponseBody`/`@RestController` → tries to resolve a view (404/500); wrong `Content-Type`/`Accept` → 415/406; `/actuator/mappings` to inspect routes; enable `logging.level.org.springframework.web=DEBUG`.

Interview Q / Follow-ups

- Walk through the DispatcherServlet request lifecycle.
- What are HandlerMapping / HandlerAdapter / MessageConverter?
- How does `@RequestBody` deserialize JSON? *(HttpMessageConverter / Jackson.)*
- Where do interceptors fire relative to the controller?
- How are exceptions turned into responses?

Hands-on Exercise

Register a `HandlerInterceptor` that logs request timing and add a `HandlerMethodArgumentResolver` for a custom `@CurrentUser` annotation.

Best Practices

Thin controllers (delegate to services); DTOs + validation; return `ResponseEntity` or `ProblemDetail`; keep blocking work off request threads when heavy.

4.2 Controllers, REST Controllers & Request Mapping

- `@Controller` returns view names; `@RestController` = `@Controller` + `@ResponseBody` (returns serialized body).
- `@RequestMapping` (class + method); shortcuts `@GetMapping`, `@PostMapping`, `@PutMapping`, `@PatchMapping`, `@DeleteMapping`.

- Params: `@PathVariable`, `@RequestParam`, `@RequestBody`, `@RequestHeader`, `@RequestPart` (multipart), `@ModelAttribute`.
- `produces` / `consumes` for content negotiation; `@ResponseStatus` for codes.

Interview Q

- `@RequestParam` vs `@PathVariable` vs `@RequestBody`.
- `@Controller` vs `@RestController`.
- How to return a specific status + headers? *(`ResponseEntity`.)*

4.3 Validation in MVC

`@Valid` / `@Validated` on `@RequestBody` / `@ModelAttribute` → binding errors → `MethodArgumentNotValidException` (body) / `BindException` (form) / `ConstraintViolationException` (method params on `@Validated` beans). Handle centrally (Module 3.6). See also Module 14.

4.4 Content Negotiation & Message Converters

Core Concept

Spring picks the response representation using the `Accept` header (and/or URL suffix/param). `HttpMessageConverter`s translate between Java objects and wire formats: - `MappingJackson2HttpMessageConverter` (JSON, default) - `MappingJackson2XmlHttpMessageConverter` / JAXB (XML) - `StringHttpMessageConverter`, `ByteArrayHttpMessageConverter`, form/multipart

ASCII

```
request Accept: application/json
  -> ContentNegotiationManager picks JSON
  -> RequestMappingHandlerAdapter selects Jackson converter
  -> object serialized to JSON
```

Interview Q / Follow-ups

- How does content negotiation decide the format?
- How does `@RequestBody` / `@ResponseBody` use converters?
- Customize Jackson (dates, naming, null handling)? *(`Jackson2ObjectMapperBuilderCustomizer`, `@JsonProperty`, `@JsonFormat`.)*
- 406 vs 415 — which is which? *(406 = can't produce `Accept`; 415 = can't consume `Content-Type`.)*

Module 4 — One-Page Cheat Sheet

Component	Role
DispatcherServlet	front controller orchestrating everything
HandlerMapping	URL+method → handler
HandlerAdapter	invokes handler
ArgumentResolver	binds params (@RequestBody/@PathVariable/…)
HttpMessageConverter	JSON/XML ↔ objects (Jackson)
HandlerInterceptor	pre/post/afterCompletion
ViewResolver	view-based responses (non-REST)
HandlerExceptionResolver	exceptions → responses

Flow: filters → dispatcher → mapping → preHandle → adapter(args+converter) → controller → converter/view → postHandle → afterCompletion → response.

Module 4 — Top Interview Questions

1. Full DispatcherServlet request lifecycle (the classic).
2. HandlerMapping vs HandlerAdapter.
3. How `@RequestBody` / `@ResponseBody` work (message converters).
4. Content negotiation; 406 vs 415.
5. Filter vs interceptor placement.
6. `@Controller` vs `@RestController`.
7. Where and how validation errors surface.
8. How exceptions become HTTP responses.
9. `@RequestParam` vs `@PathVariable`.
10. Thread-per-request model implications.

Module 4 — Common Mistakes

- Forgetting `@ResponseBody` / `@RestController` (view resolution attempted).
- Wrong `consumes` / `produces` → 415/406.
- Business logic in controllers.
- Not handling `MethodArgumentNotValidException` globally.

Module 4 — Mock Interview

1. *"Trace `POST /orders` with a JSON body end to end."* → filters → dispatcher → mapping → preHandle → adapter → Jackson deserialize → @Valid → controller → service → Jackson serialize → 201 → afterCompletion.
2. *"Client gets 406."* → server can't produce the requested `Accept` type.
3. *"How do I add a per-request correlation id visible in all logs?"* → servlet filter sets MDC early (before dispatcher) and clears it after.
4. *"How does Spring turn my `Order` object into JSON?"* → `MappingJackson2HttpMessageConverter` selected by content negotiation.

5. *“When would you pick WebFlux over MVC?”* → very high concurrency, mostly I/O-bound, need backpressure/non-blocking clients.

Next → Module 5: Spring Data JPA & Hibernate.

Module 5 — Spring Data JPA & Hibernate

> Highest priority. This is where senior candidates are separated: interviewers ask about the **persistence context, dirty checking, lazy loading, the N+1 problem, transactions, and the SQL Hibernate actually generates.** Always be ready to reason about the emitted SQL and its performance.

5.1 Entity Lifecycle & Persistence Context

Why Interviewers Ask This

The persistence context (a.k.a. first-level cache) explains almost every "why did Hibernate do that?" behavior: dirty checking, identity, lazy loading, `LazyInitializationException`.

Core Concept — Entity States

```
NEW/TRANSIENT ---persist()---> MANAGED ---commit/flush---> DB
     ^                          |   ^
     |                     detach()  merge()
     |                          v   |
  (gc)                       DETACHED
                              |
                            remove() -> REMOVED
```

- **Transient** — new object, not associated, no DB row.
- **Managed/Persistent** — attached to the persistence context; changes are tracked and auto-flushed.
- **Detached** — was managed, context closed; changes NOT tracked.
- **Removed** — scheduled for delete.

Persistence Context = First-Level Cache

Per-`EntityManager`/transaction map of `entityId → entity instance`. Guarantees **identity** (same id → same object within the context), enables **dirty checking**, and batches SQL until **flush**.

ASCII

```
Transaction
┌──────────────────────── Persistence Context (L1 cache) ──────────────────┐
│  id=1 -> User@abc (managed)     id=2 -> Order@def (managed)              │
│  tracks snapshots for dirty checking; dedups reads by id                 │
└──────────────────────────────────────────────────────────────────────────┘
      flush (before commit / query) -> generates INSERT/UPDATE/DELETE
```

Interview Q / Follow-ups

- Explain entity states and transitions.
- What is the persistence context / L1 cache; is it shared across requests? *(No — per EntityManager/transaction.)*
- `persist` vs `merge` vs `save`? `save` vs `saveAndFlush`?
- Why does `LazyInitializationException` happen? *(access lazy field after context/transaction closed.)*

5.2 Dirty Checking & Flush

Core Concept

For managed entities, Hibernate keeps a **snapshot** at load time. At flush (before commit or before a query that could be affected), it compares current vs snapshot and auto-generates `UPDATE` for changed fields — **you never call `update()`**.

```
@Transactional
void raise(Long id) {
    Employee e = repo.findById(id).orElseThrow();
    e.setSalary(e.getSalary().multiply(BigDecimal.valueOf(1.1)));
    // no save() needed — dirty checking flushes UPDATE at commit
}
```

FlushMode

`AUTO` (default): flush before commit and before queries; `COMMIT`: only at commit. `flush()` ≠ `commit()` (flush syncs SQL; commit ends the transaction).

Common Mistakes / Performance

Modifying detached entities expecting auto-update; unnecessary `saveAndFlush` in loops; huge persistence contexts (memory + slow dirty-check) → clear/batch.

Interview Q

- What is dirty checking and when does flush happen?
- Difference between flush and commit.
- Why did an UPDATE fire even though I never called save?

5.3 Caching: First vs Second Level

	L1 (persistence context)	L2 (shared)
Scope	per EntityManager/tx	per SessionFactory (app-wide)
Default	always on	off (opt-in)
Stores	entity instances by id	entity/collection/query data
Provider	built-in	EhCache, Caffeine, Hazelcast, Redis (via provider)
Config	none	`@Cacheable` (Hibernate), `hibernate.cache.use_second_level_cache=true`

Query cache is separate and must be enabled explicitly; easy to misuse (invalidation cost). L2 helps read-mostly reference data; risky for write-heavy or multi-node consistency.

Interview Q

- L1 vs L2 cache; when enable L2? Risks with clustering (stale reads / invalidation).

5.4 Lazy vs Eager Loading, Fetch Join & the N+1 Problem

Why Interviewers Ask This

N+1 is the #1 JPA performance bug in production. They want to see you detect and fix it.

Core Concept

- **FetchType.LAZY** — association loaded on first access (proxy). Default for `@OneToMany` / `@ManyToMany`.

- **FetchType.EAGER** — loaded immediately with the owner. Default for `@ManyToOne`/`@OneToOne`. **Prefer LAZY everywhere**; fetch what you need per query.

The N+1 Problem

```
select * from orders;                 -- 1 query, returns N orders
for each order: select * from items where order_id=?;   -- N queries
=> 1 + N queries
```

Fixes

- **Fetch join** (JPQL): `select o from Order o join fetch o.items` → one query.
- `@EntityGraph` on the repository method (declarative fetch plan).
- **Batch fetching**: `@BatchSize(size=50)` / `hibernate.default_batch_fetch_size` → turns N selects into N/50 `IN (...)` selects.
- **DTO projection**: select only needed columns (avoids entities entirely).

ASCII

```
Bad:   [orders] --then per row--> [items][items][items]...  (1+N)
Good:  [orders JOIN FETCH items]  ->  single result set     (1)
```

Real Production Example

An endpoint listing 200 orders each with items ran 201 queries and timed out. Switching to `@EntityGraph(attributePaths="items")` cut it to 1 query; p99 dropped from 3s to 40ms.

Common Mistakes / Trade-offs

- `EAGER` everywhere → loads huge graphs, hidden N+1.
- **`join fetch` + pagination** → Hibernate paginates in memory (`HHH000104` warning) and can duplicate rows; use `@EntityGraph` or two queries, or fetch ids then entities.
- Multiple `join fetch` on collections → cartesian product.

Debugging

`spring.jpa.show-sql=true` + `hibernate.format_sql`; better: `p6spy` or datasource-proxy to count queries; enable `logging.level.org.hibernate.SQL=DEBUG` and bind params `org.hibernate.orm.jdbc.bind=TRACE`. Assert query counts in tests.

Interview Q / Follow-ups

- What is the N+1 problem; how do you detect and fix it?
- LAZY vs EAGER; defaults per association type.
- `join fetch` vs `@EntityGraph`; the pagination pitfall.
- Why prefer DTO projections for read APIs?

Hands-on Exercise

Reproduce N+1 with `@OneToMany` LAZY, confirm 1+N queries in logs, then fix with `@EntityGraph` and verify a single query.

5.5 Cascade Types & Orphan Removal

`CascadeType`: `PERSIST, MERGE, REMOVE, REFRESH, DETACH, ALL`. Propagate operations from parent to children. `orphanRemoval=true` deletes children removed from the collection.

Common Mistakes

`CascadeType.ALL` on a `@ManyToOne` to a shared parent (deletes shared data!); confusing `orphanRemoval` with `REMOVE`.

Interview Q

When to use cascade; `orphanRemoval` vs `CascadeType.REMOVE`.

5.6 Transactions (@Transactional)

Core Concept

`@Transactional` (Spring, AOP-proxy based) demarcates a transaction around a method: begin → run → commit, or rollback on unchecked exception.

Key attributes

- **propagation**: `REQUIRED` (default — join or start), `REQUIRES_NEW` (suspend + new), `NESTED` (savepoint), `SUPPORTS`, `MANDATORY`, `NEVER`, `NOT_SUPPORTED`.
- **isolation**: `READ_COMMITTED` (typical default), `REPEATABLE_READ`, `SERIALIZABLE` (see Module 10).
- **rollbackFor**: by default rolls back on `RuntimeException`/`Error` only — **checked exceptions do NOT roll back** unless `rollbackFor` is set.
- **readOnly=true**: hint (flush mode MANUAL, may skip dirty checking) — optimize read paths.
- **timeout**.

Lifecycle / ASCII

```
caller -> [proxy] begin tx (get connection, autocommit=false)
                -> target method runs
                -> commit  (or rollback on RuntimeException)
          [proxy] release connection
self-invocation (this.method()) bypasses proxy => NO transaction!
non-public methods => advice not applied (default proxy).
```

Real Production Example

Order + payment must be atomic:

```
@Transactional
public void placeOrder(Order o) {
    orderRepo.save(o);
    paymentClient.charge(o);        // if this throws RuntimeException -> rollback
}
```

For the **outbox pattern**, write the domain change and an outbox row in the same transaction (Module 7).

Common Mistakes (very common in interviews)

- Self-invocation / private / final methods → no transaction.
- Expecting checked exceptions to roll back (they don't by default).
- Calling remote/HTTP inside a long transaction → holds DB connection → pool exhaustion.
- `@Transactional` on the class but a public method calling another public method in the same bean expecting `REQUIRES_NEW` (bypassed).

Interview Q / Follow-ups

- How does `@Transactional` work internally (proxy/AOP)?
- Propagation levels — `REQUIRED` vs `REQUIRES_NEW` vs `NESTED`.
- Which exceptions roll back by default? How to change it?

- Why does self-invocation break it?
- What does `readOnly=true` actually do?

Hands-on Exercise

Demonstrate that a checked exception does NOT roll back, then add `rollbackFor = Exception.class` and observe the rollback.

5.7 JPQL, Criteria API, Specifications, Pagination

- **JPQL** — object-oriented query language over entities: `select o from Order o where o.status = :s`. `@Query` in repositories; `nativeQuery=true` for raw SQL.
- **Criteria API** — programmatic, type-safe (metamodel) query building; verbose but good for complex dynamic queries.
- **Specifications** (Spring Data) — composable predicates (`spec.and(other)`) for dynamic filters; cleaner than Criteria for search endpoints.
- **Derived queries** — `findByStatusAndCreatedAfter(...)`.
- **Pagination** — `Pageable` / `Page` / `Slice`; `Page` runs a count query, `Slice` doesn't (cheaper when you only need "next").

Interview Q

- JPQL vs native SQL vs Criteria vs Specifications — when each?
- `Page` vs `Slice` (extra count query).
- Keyset (seek) pagination vs offset for large tables (offset gets slow deep in).

5.8 Optimistic vs Pessimistic Locking

	Optimistic	Pessimistic
Mechanism	`@Version` column; check-and-set at commit	DB row lock (`SELECT ... FOR UPDATE`)
Conflict	`OptimisticLockException` on stale version	blocks/waits
Best for	low contention, high throughput	high contention, must not lose updates
Cost	retries on conflict	lock contention, deadlock risk
JPA	`@Version`	`@Lock(PESSIMISTIC_WRITE)`

Real Production Example

Inventory decrement under concurrency: optimistic `@Version` with retry for most cases; pessimistic `FOR UPDATE` for hot single rows (e.g. a shared counter) to serialize.

Interview Q / Follow-ups

- Optimistic vs pessimistic; how `@Version` works.
- How to handle `OptimisticLockException`? *(retry / surface conflict 409.)*
- Lost update problem and how each lock prevents it.

Module 5 — One-Page Cheat Sheet

Topic	Key point
Entity states	transient/managed/detached/removed
L1 cache	per-tx persistence context; identity + dirty checking
Dirty checking	snapshot compare at flush → auto UPDATE (no save needed)
flush vs commit	flush syncs SQL; commit ends tx
Fetch	LAZY by default for collections; EAGER for *ToOne — prefer LAZY
N+1	fetch join / @EntityGraph / @BatchSize / DTO projection
join fetch + paging	in-memory paging pitfall (HHH000104) → @EntityGraph
Cascade	ALL/REMOVE/orphanRemoval — beware shared parents
@Transactional	proxy-based; RuntimeException rolls back; self-invocation bypasses
Propagation	REQUIRED vs REQUIRES_NEW vs NESTED
Locking	optimistic @Version (retry) vs pessimistic FOR UPDATE
Pagination	Page(count) vs Slice; keyset for deep pages

Module 5 — Top Interview Questions

1. Explain the persistence context and dirty checking; when does flush happen?
2. What is the N+1 problem — detect and fix (fetch join, EntityGraph, batch size).
3. LAZY vs EAGER; cause of `LazyInitializationException`.
4. How does `@Transactional` work; which exceptions roll back by default?
5. Propagation levels, especially `REQUIRES_NEW` vs `NESTED`.
6. Optimistic vs pessimistic locking; `@Version`.
7. `save` vs `persist` vs `merge`; `Page` vs `Slice`.
8. First vs second level cache; risks of L2 in a cluster.
9. `join fetch` + pagination pitfall.
10. When to use DTO projections / native queries.

Module 5 — Common Mistakes

- EAGER everywhere → hidden N+1 and huge graphs.
- Expecting detached entity changes to persist.
- Remote calls inside a transaction (connection held → pool exhaustion).
- Checked exception not rolling back (default).
- `CascadeType.ALL` deleting shared parents.
- `join fetch` with `Pageable` (in-memory pagination).

Module 5 — Mock Interview

1. *"An endpoint runs 201 queries."* → classic N+1; show detection (SQL logs / query counter) and fix with `@EntityGraph`/fetch join/batch size.
2. *"`LazyInitializationException` in the JSON serializer."* → lazy access after tx closed; fix with fetch join/EntityGraph or a DTO — NOT `OpenSessionInView` (anti-pattern).

3. *“Two users update inventory concurrently and one loses the update.”* → lost update; add `@Version` (optimistic) or `FOR UPDATE` (pessimistic).
4. *“Rollback isn’t happening on our checked `PaymentDeclinedException`.”* → default only rolls back unchecked; add `rollbackFor`.
5. *“DB connection pool exhausted under load.”* → transactions held open across remote calls; shorten transactions, move I/O outside, add timeouts.

Next → Module 6: Spring Security.

Module 6 — Spring Security

> Spring Security is a chain of servlet filters. The must-know answer is the **complete authentication flow and the Security Filter Chain**. Interviewers also drill JWT vs sessions, OAuth2, and CSRF/CORS.

6.1 Core Concepts: Authentication vs Authorization

- **Authentication (AuthN)** — *who are you?* Verify identity (credentials, token).
- **Authorization (AuthZ)** — *what can you do?* Check permissions/roles.
- Key abstractions: `Authentication` (principal + authorities), `SecurityContext` (holds the current `Authentication`, stored in `SecurityContextHolder`, ThreadLocal by default), `UserDetails` / `UserDetailsService`, `AuthenticationManager` / `AuthenticationProvider`, `GrantedAuthority`.

6.2 The Security Filter Chain (the classic question)

Why Interviewers Ask This

Security “magic” is just an ordered filter chain. Naming the key filters proves real understanding.

Core Concept

A `FilterChainProxy` (`springSecurityFilterChain`) delegates to an ordered list of `SecurityFilter`s. Each request passes through them before reaching the DispatcherServlet.

Key filters (order matters)

```
Request
  │
  ▼
SecurityContextPersistenceFilter / SecurityContextHolderFilter
  │   (load SecurityContext from session/repo)
  ▼
CorsFilter -> CsrfFilter
  ▼
(Authentication filters)
  UsernamePasswordAuthenticationFilter   (form login /login POST)
  BearerTokenAuthenticationFilter        (OAuth2 resource server / JWT)
  BasicAuthenticationFilter              (HTTP Basic)
  │
  ▼
ExceptionTranslationFilter   (catches AuthN/AuthZ exceptions ->
                              401 entry point / 403)
  │
  ▼
AuthorizationFilter (was FilterSecurityInterceptor)  (authorize request)
  │
  ▼
DispatcherServlet -> Controller
```

Authentication Flow (username/password)

```
1. UsernamePasswordAuthenticationFilter builds an unauthenticated
   UsernamePasswordAuthenticationToken(username, password)
2. -> AuthenticationManager (ProviderManager)
3. -> AuthenticationProvider (DaoAuthenticationProvider)
4.     -> UserDetailsService.loadUserByUsername()
5.     -> PasswordEncoder.matches(raw, stored)
6. success: build authenticated Authentication (with authorities)
7. store in SecurityContextHolder (and session, if stateful)
8. on failure: AuthenticationException -> 401 via entry point
```

ASCII — AuthN Components

```
 filter -> AuthenticationManager (ProviderManager)
             -> [ DaoAuthenticationProvider ] -> UserDetailsService + PasswordEncoder
             -> [ JwtAuthenticationProvider ] -> validate signature/claims
          returns authenticated Authentication -> SecurityContext
```

Real Production Example (modern lambda DSL, Spring Security 6)

```
@Bean
SecurityFilterChain chain(HttpSecurity http) throws Exception {
  http.csrf(csrf -> csrf.disable())                // stateless API
      .cors(Customizer.withDefaults())
      .sessionManagement(s -> s.sessionCreationPolicy(STATELESS))
      .authorizeHttpRequests(a -> a
          .requestMatchers("/public/**", "/actuator/health").permitAll()
          .requestMatchers("/admin/**").hasRole("ADMIN")
          .anyRequest().authenticated())
      .oauth2ResourceServer(o -> o.jwt(Customizer.withDefaults()));
  return http.build();
}
```

Interview Q / Follow-ups

- Walk through the security filter chain / authentication flow.
- `AuthenticationManager` vs `AuthenticationProvider` vs `UserDetailsService`.
- Where is the authenticated user stored? *(SecurityContextHolder → ThreadLocal.)*
- What does `ExceptionTranslationFilter` do? *(AuthN → 401 entry point; AuthZ → 403.)*
- How to add a custom filter? *(`addFilterBefore/After`.)*

Hands-on Exercise

Add a custom `OncePerRequestFilter` that reads an API key header and populates the `SecurityContext` before `AuthorizationFilter`.

6.3 Password Encoding

Store **hashed + salted** passwords. Use `BCryptPasswordEncoder` (adaptive, salted) or `Argon2` / `PBKDF2`. `DelegatingPasswordEncoder` (default) prefixes the algorithm `{bcrypt}...` enabling migration. **Never** store plaintext or use fast/unsalted hashes (MD5/SHA-1).

Interview Q

Why BCrypt over SHA-256? *(slow, salted, adaptive work factor resists brute force.)* How to rotate encoders? *(DelegatingPasswordEncoder.)*

6.4 JWT, Sessions, OAuth2 & OpenID Connect

Session vs JWT

	Session (stateful)	JWT (stateless)
Server state	session store (memory/Redis)	none (token self-contained)
Scaling	needs shared session store / sticky	scales horizontally easily
Revocation	easy (delete session)	hard (until expiry) → short TTL + refresh + denylist
Payload	opaque id	signed claims (sub, roles, exp)
Use	classic web apps	APIs, microservices, SPAs/mobile

JWT structure

`header.payload.signature` (Base64URL). Signature (HMAC or RSA/EC) proves integrity; **payload is not encrypted** — don't put secrets in it. Validate: signature, `exp`, `iss`, `aud`.

OAuth2 roles & flow

- Roles: **Resource Owner** (user), **Client** (app), **Authorization Server** (issues tokens), **Resource Server** (your API).
- **Authorization Code + PKCE** is the recommended flow for web/mobile/SPA.
- **Client Credentials** for service-to-service.
- **OpenID Connect (OIDC)** layers *authentication* on OAuth2 by adding an **ID token** (a JWT with user identity claims). OAuth2 alone = authorization; OIDC adds login/identity.

ASCII — Authorization Code + PKCE

```
User -> Client: login
Client -> AuthServer: /authorize (code_challenge)
User authenticates at AuthServer
AuthServer -> Client: authorization code (redirect)
Client -> AuthServer: /token (code + code_verifier)
AuthServer -> Client: access token (+ id token, refresh)
Client -> Resource Server: request + Bearer access token
Resource Server: validate JWT (jwks) -> serve
```

Real Production Example

Microservices behind an API gateway: the gateway/authorization server issues JWTs; each service is an **OAuth2 resource server** validating the JWT signature against the JWKS endpoint (`spring.security.oauth2.resourceserver.jwt.jwk-set-uri`) and mapping claims → authorities. No shared session store needed.

Common Mistakes / Trade-offs

- Long-lived JWTs with no revocation strategy.
- Putting sensitive data in JWT payload (readable).
- `alg=none` / not validating signature.
- Storing JWT in `localStorage` (XSS risk) vs httpOnly cookie (CSRF considerations).

Interview Q / Follow-ups

- Session vs JWT — trade-offs, when each?
- How do you revoke a JWT? *(short TTL + refresh token rotation + denylist.)*
- Explain OAuth2 authorization code + PKCE; why PKCE?
- OAuth2 vs OIDC?
- Where to store tokens on the client?

6.5 Method Security

`@EnableMethodSecurity` → `@PreAuthorize("hasRole('ADMIN')")`, `@PostAuthorize`, `@PreFilter`/`@PostFilter`, SpEL with `#params` and `authentication`. Enforces AuthZ at the service layer, not just URLs.

Interview Q

URL-based vs method security; `hasRole` vs `hasAuthority` (ROLE_ prefix).

6.6 CSRF & CORS

CSRF (Cross-Site Request Forgery)

An attacker tricks a logged-in browser into making a state-changing request using its cookies. Defense: **synchronizer token** (Spring's `CsrfFilter`) or SameSite cookies. **Stateless JWT-in-header APIs are not vulnerable** to classic CSRF (no ambient cookie), so CSRF is commonly disabled for them — but *cookie-based* auth needs CSRF protection.

CORS (Cross-Origin Resource Sharing)

Browser same-origin policy blocks cross-origin JS calls unless the server sends `Access-Control-Allow-*` headers. Configure allowed origins/methods/headers; handle preflight `OPTIONS`. CORS is a *browser* mechanism, not a security boundary.

ASCII — CSRF vs CORS

```
CSRF: browser auto-sends cookies -> server must verify a token it can't forge
CORS: browser blocks cross-origin reads unless server opts in via headers
```

Interview Q / Follow-ups

- What is CSRF; why can you disable it for a stateless JWT API but not a cookie one?
- CORS preflight — when does it happen? *(non-simple requests: custom headers, PUT/DELETE, etc.)*
- Is CORS a security feature? *(browser-enforced; not server-side authorization.)*

Module 6 — One-Page Cheat Sheet

Topic	Key point
Filter chain	ordered filters before DispatcherServlet
AuthN flow	filter → AuthenticationManager → Provider → UserDetailsService + PasswordEncoder → SecurityContext
Context	SecurityContextHolder (ThreadLocal)
Passwords	BCrypt/Argon2, salted, DelegatingPasswordEncoder
Session vs JWT	stateful+revocable vs stateless+scalable
JWT	header.payload.signature; payload not encrypted; validate exp/iss/aud
OAuth2	code+PKCE (web), client-credentials (service); resource server validates JWT via JWKS
OIDC	adds ID token (identity) on top of OAuth2
Method security	@PreAuthorize SpEL
CSRF	needed for cookie auth; skip for header-token stateless
CORS	browser opt-in via Allow-* headers; not authz

Module 6 — Top Interview Questions

1. Walk through the security filter chain and authentication flow.
2. `AuthenticationManager` vs `Provider` vs `UserDetailsService`.
3. Session vs JWT; how to revoke a JWT.
4. OAuth2 authorization code + PKCE; OAuth2 vs OIDC.
5. Why BCrypt; how does salting work.
6. CSRF — what is it and when to enable/disable.
7. CORS vs CSRF; preflight requests.
8. URL vs method security; `@PreAuthorize`.
9. How to add a custom authentication filter.
10. How a resource server validates a JWT (JWKS).

Module 6 — Common Mistakes

- Disabling CSRF on a cookie-based app.
- Long-lived, non-revocable JWTs; secrets in payload.
- Treating CORS as authorization.
- Storing plaintext / fast-hashed passwords.
- Confusing 401 (unauthenticated) with 403 (unauthorized).

Module 6 — Mock Interview

1. *"Explain what happens from `POST /login` to a populated SecurityContext."* → UsernamePasswordAuthenticationFilter → AuthenticationManager → DaoAuthenticationProvider → UserDetailsService + PasswordEncoder → SecurityContext(+session).
2. *"We use JWTs; how do you log a user out immediately?"* → short TTL + refresh rotation + server-side denylist keyed by jti.
3. *"Frontend gets a CORS error."* → configure allowed origin/methods/headers + preflight; note it's browser-enforced.
4. *"Service-to-service auth in microservices?"* → OAuth2 client-credentials; each service a resource server validating JWT via JWKS.
5. *"401 vs 403?"* → 401 not authenticated (who are you); 403 authenticated but not permitted.

Next → Module 7: Microservices.

Module 7 — Microservices

> Highest priority for senior roles. Interviewers care less about “what is a microservice” and more about **resilience patterns, distributed data (saga/outbox), idempotency, and communication trade-offs**. Bring concrete failure scenarios.

7.1 Monolith vs Microservices

Core Concept

- **Monolith** — one deployable, one codebase, one DB. Simple to build/deploy/ debug; strong consistency; scales as a whole.
- **Microservices** — many small, independently deployable services, each owning its data, communicating over the network.

Aspect	Monolith	Microservices
Deploy	one unit	independent per service
Scaling	whole app	per service
Data	shared DB, ACID	DB-per-service, eventual consistency
Team	coupled	autonomous teams
Complexity	low ops, high code coupling	high ops (network, tracing, saga)
Failure	in-process	partial failures, network

Trade-offs / Common Mistakes

Microservices trade code complexity for **distributed-systems complexity** (network, partial failure, data consistency, observability). Anti-patterns: distributed monolith (services deploy together / share a DB), nano-services, shared database across services. **Start with a modular monolith**; extract services when scaling/team boundaries demand it.

Interview Q / Follow-ups

- When would you NOT use microservices?
- What is a distributed monolith and why is it bad?
- How do you decide service boundaries? *(bounded contexts / DDD.)*

7.2 API Gateway

Core Concept

Single entry point for clients. Handles cross-cutting concerns: routing, auth/token validation, rate limiting, request aggregation, TLS termination, CORS, load balancing. Spring Cloud Gateway (reactive) is the common choice.

ASCII

```
clients -> [ API GATEWAY ]  (auth, rate-limit, routing, aggregation)
                |-> order-service
                |-> user-service
                |-> payment-service
```

Trade-offs

Central point of failure/latency → run HA, keep logic thin. Distinguish from a **service mesh** (sidecar, east-west traffic, mTLS) vs gateway (north-south).

Interview Q

Gateway responsibilities; gateway vs load balancer vs service mesh; BFF pattern.

7.3 Service Discovery & Config Server

Service Discovery

Services register with a registry (Eureka, Consul, or Kubernetes DNS/Services) so callers resolve instances dynamically instead of hardcoding hosts. - **Client-side** (Eureka + Ribbon/Spring Cloud LoadBalancer): client picks an instance. - **Server-side** (K8s Service, cloud LB): infra routes.

Config Server

Centralized, versioned external config (Spring Cloud Config backed by Git); services fetch config at startup; refresh via `@RefreshScope` / bus. In K8s, ConfigMaps/Secrets often replace it.

ASCII

```
service starts -> register in Eureka -> heartbeat
caller -> discovery -> instance list -> load-balance -> call
```

Interview Q

Client vs server-side discovery; how config changes propagate; secrets handling.

7.4 Resilience: Circuit Breaker, Retry, Timeout, Bulkhead

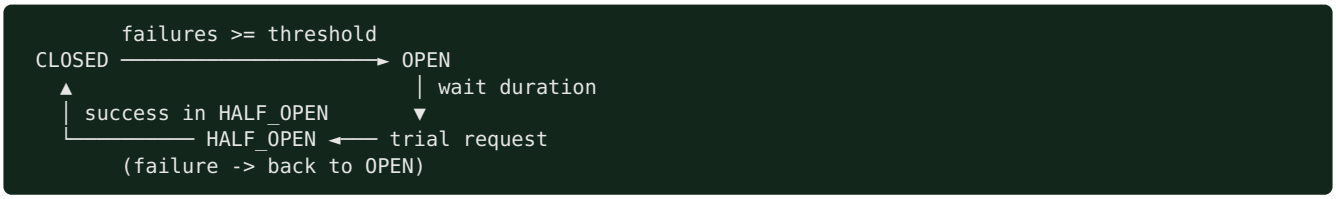
Why Interviewers Ask This

Partial failure is the defining trait of distributed systems. They want the four core patterns and how they interact (Resilience4j in Spring).

Patterns

- **Timeout** — never wait forever; bound every remote call. *First line of defense.*
- **Retry** — retry transient failures, with **exponential backoff + jitter**; only for **idempotent** ops; cap attempts.
- **Circuit Breaker** — stop calling a failing dependency; states **CLOSED → (failure rate exceeds threshold) → OPEN → (after wait) HALF_OPEN → CLOSED**. Fails fast + fallback while OPEN, giving the dependency time to recover.
- **Bulkhead** — isolate resources (separate thread pools / concurrency limits per dependency) so one slow dependency can't consume all threads (like ship compartments).
- **Rate limiter / fallback** round out the toolkit.

ASCII — Circuit Breaker States



Real Production Example (Resilience4j)

```
@CircuitBreaker(name = "pricing", fallbackMethod = "cachedPrice")
@Retry(name = "pricing")
@TimeLimiter(name = "pricing")
public CompletableFuture<Price> getPrice(String sku) { ... }

Price cachedPrice(String sku, Throwable t) { return cache.getLast(sku); }
```

Common Mistakes / Trade-offs

- Retrying non-idempotent operations (double charge).
- Retry storms amplifying an outage (no backoff/jitter, no circuit breaker).
- Timeouts longer than the caller's timeout (cascading).
- No fallback → circuit breaker just fails fast with errors.

Interview Q / Follow-ups

- Explain circuit breaker states.
- Why combine retry + circuit breaker + timeout? Order of interaction?
- Why must retried operations be idempotent?
- What is a bulkhead and what problem does it solve?
- What is a retry storm / thundering herd; how to prevent (backoff + jitter)?

Hands-on Exercise

Wrap a flaky client with Resilience4j `@Retry` (exp backoff+jitter) + `@CircuitBreaker` + fallback; trip the breaker and observe HALF_OPEN recovery.

7.5 Distributed Transactions, Saga & Outbox

Why Interviewers Ask This

DB-per-service kills 2PC/XA in practice. They want the **saga** pattern and how to publish events reliably (**outbox**).

The problem

No single ACID transaction spans services. 2PC (XA) is blocking, doesn't scale, and couples services. Instead use **eventual consistency** with sagas.

Saga Pattern

A saga = a sequence of local transactions, each publishing an event that triggers the next; failures trigger **compensating transactions** (semantic undo). - **Choreography** — services react to each other's events (no central coordinator). Simple, but logic is spread out; hard to track. - **Orchestration** — a central orchestrator tells each service what to do and handles compensation. Clearer, testable; orchestrator is a dependency.

ASCII — Order Saga (orchestration)

```
Orchestrator: create order (pending)
  -> reserve inventory  (ok?)
      -> charge payment (ok?)
          -> confirm order
          (payment fails) -> compensate: release inventory -> cancel order
```

Outbox Pattern (reliable event publishing)

Problem: you must update the DB **and** publish an event atomically, but DB and broker are separate systems (dual-write problem). Solution: in the **same local transaction**, write the domain change and an `outbox` row. A separate

relay (polling or CDC via Debezium) reads the outbox and publishes to Kafka, then marks it sent. Guarantees at-least-once publish consistent with the DB.

```
@Transactional { orderRepo.save(order); outboxRepo.save(event); }  // atomic
relay/CDC -> read outbox -> publish to Kafka -> mark sent
```

Idempotency

Because delivery is at-least-once, consumers **must be idempotent**: dedup on a business/message key (`idempotency-key`, `eventId`), use `INSERT ... ON CONFLICT`, upserts, or a processed-ids table. Essential for retries, saga steps, and payment APIs.

Interview Q / Follow-ups

- Why not 2PC in microservices?
- Saga: choreography vs orchestration — trade-offs.
- How do you publish events reliably? *(outbox / CDC — solves dual-write.)*
- How do you make a consumer idempotent?
- What is a compensating transaction? Example.

Hands-on Exercise

Design an order saga with orchestration + compensations; add an outbox table and a relay; make the payment consumer idempotent via an `idempotency_key` unique constraint.

7.6 Service Communication: REST vs gRPC & Event-Driven

Sync vs Async

- **Synchronous (REST/gRPC)** — request/response, temporal coupling; simpler, but the caller waits and failures propagate.
- **Asynchronous (events/messaging)** — publish events, consumers react; decoupled, resilient, scalable, but eventually consistent and harder to trace.

REST vs gRPC

	REST/JSON	gRPC
Transport	HTTP/1.1 (or 2)	HTTP/2
Payload	JSON (text)	Protobuf (binary, compact)
Contract	OpenAPI (loose)	`.proto` (strict, codegen)
Streaming	limited (SSE)	bidirectional streaming
Perf	good	higher throughput, lower latency
Browser	native	needs grpc-web
Use	public APIs, simplicity	internal high-perf service-to-service

Event-Driven Architecture (EDA)

Services emit domain events to a broker (Kafka); consumers react. Enables decoupling, replay, CQRS, event sourcing. Trade-off: eventual consistency, ordering, debugging across async flows.

Interview Q / Follow-ups

- REST vs gRPC — when each?
- Sync vs async communication trade-offs.
- What is EDA; benefits and challenges?

- How do you trace a request across async boundaries? *(correlation/trace id propagation — Module 11.)*

Module 7 — One-Page Cheat Sheet

Topic	Key point
Monolith vs micro	micro trades code coupling for distributed complexity; avoid distributed monolith
Gateway	north-south entry: routing/auth/rate-limit/aggregation
Discovery	Eureka/Consul/K8s DNS; client vs server-side
Config	Spring Cloud Config (Git) / ConfigMaps
Timeout	bound every remote call (first defense)
Retry	idempotent only; exp backoff + jitter; cap
Circuit breaker	CLOSED→OPEN→HALF_OPEN; fail fast + fallback
Bulkhead	isolate pools per dependency
Saga	local txns + compensations; choreography vs orchestration
Outbox	atomic DB write + event row; relay/CDC publishes (fixes dual-write)
Idempotency	dedup on key; at-least-once needs it
REST vs gRPC	JSON/HTTP1 vs protobuf/HTTP2 streaming

Module 7 — Top Interview Questions

1. How do you handle transactions across microservices? (saga, why not 2PC)
2. Explain circuit breaker states; combine with retry + timeout.
3. Outbox pattern and the dual-write problem.
4. Why must retried/consumed operations be idempotent; how to implement.
5. Choreography vs orchestration saga.
6. REST vs gRPC; sync vs async trade-offs.
7. API gateway responsibilities; gateway vs service mesh.
8. Service discovery mechanisms.
9. How to prevent retry storms / cascading failures.
10. How do you define service boundaries?

Module 7 — Common Mistakes

- Sharing a database across services (distributed monolith).
- Retrying non-idempotent operations.
- Dual-write without outbox (lost events).
- No timeouts → cascading failures / thread exhaustion.
- Circuit breaker without a fallback.

Module 7 — Mock Interview

1. *"Order needs inventory + payment across services — how do you keep them consistent?"* → orchestration saga with compensations; outbox for reliable events; idempotent consumers.

2. *“A downstream dependency is slow and taking your service down.”* → timeouts + bulkhead + circuit breaker + fallback.
3. *“You publish an event after committing the DB, but sometimes the event is lost.”* → dual-write; use outbox/CDC.
4. *“A consumer processed the same payment twice.”* → at-least-once delivery; add idempotency key + unique constraint / dedup store.
5. *“Internal services need very low latency, high throughput.”* → gRPC over protobuf/HTTP2; keep REST for public APIs.

Next → Module 8: Messaging (Kafka & RabbitMQ).

Module 8 — Messaging (Kafka & RabbitMQ)

> Interviewers probe delivery guarantees (at-least/at-most/exactly-once), ordering, consumer groups, offsets, retries, and DLQs. Know **when Kafka vs RabbitMQ** and how each achieves ordering and reliability.

8.1 Kafka vs RabbitMQ — the fundamental difference

	Kafka	RabbitMQ
Model	distributed **commit log**	traditional **message broker/queue**
Consumption	pull; offset-based; **messages retained**	push; **removed on ack**
Ordering	per-partition	per-queue
Replay	yes (re-read offsets)	no (once acked, gone)
Throughput	very high (millions/s)	high, lower than Kafka
Routing	topic/partition	exchanges (direct/topic/fanout/headers)
Best for	event streaming, logs, high volume, replay, CQRS/ES	task queues, complex routing, RPC, per-message TTL/priority

One-liner: Kafka is a durable, replayable *log* optimized for throughput and streaming; RabbitMQ is a flexible *broker* optimized for routing and per-message delivery semantics.

8.2 Kafka core concepts

Producer / Topic / Partition / Offset

- **Topic** — named stream, split into **partitions** (unit of parallelism & ordering).
- **Partition** — ordered, append-only log; each message has a monotonically increasing **offset**.
- **Producer** — writes to a partition; the **partitioner** picks the partition (by `key` hash, or round-robin if no key). **Same key → same partition → ordering.**
- **Offset** — position of a message in a partition; consumers track their offset.

Consumer & Consumer Groups

- A **consumer group** shares the work: each partition is consumed by **exactly one** consumer in the group → parallelism = min(#partitions, #consumers).
- Different groups each get **all** messages (pub/sub fan-out).
- **Rebalancing** reassigns partitions when consumers join/leave.
- **Committed offset** marks progress; on restart the group resumes from it.

ASCII — Topic / Partitions / Group

```
Topic "orders" (3 partitions)
  P0: [o0][o3][o6]  <- consumer A (group G)
  P1: [o1][o4][o7]  <- consumer B (group G)
  P2: [o2][o5][o8]  <- consumer C (group G)
key(customerId) -> hash -> fixed partition -> per-customer ordering
group H reads ALL partitions independently (fan-out)
```

Ordering

Kafka guarantees ordering **only within a partition**. For per-entity ordering, key by that entity id. Global ordering ⇒ single partition (kills parallelism). Beware: `max.in.flight.requests > 1` + retries can reorder unless `enable.idempotence=true`.

Interview Q

- What determines the partition a message goes to?
- How do consumer groups provide scaling and fan-out?
- How does Kafka guarantee ordering, and its limits?
- What triggers a rebalance; why can it cause pauses/duplicates?

8.3 Delivery Guarantees: At-most / At-least / Exactly-once

Guarantee	Behavior	How
At-most-once	may lose, never duplicate	commit offset **before** processing / fire-and-forget
At-least-once	never lose, may duplicate	commit offset **after** processing (default; needs idempotent consumer)
Exactly-once	no loss, no dup	Kafka idempotent producer + **transactions** (read-process-write) or idempotent consumer + dedup

Kafka exactly-once (EOS)

`enable.idempotence=true` (dedup producer retries) + transactional producer (`transactional.id`, `initTransaction/commitTransaction`) + consumer `isolation.level=read_committed`. True EOS holds **within Kafka** (topic→topic). When a side effect hits an external DB, you still need **idempotency** (Module 7).

Producer acks

- `acks=0` (fire-and-forget, may lose), `acks=1` (leader only), `acks=all` + `min.insync.replicas` (durable, no loss on single-broker failure).

ASCII — commit timing decides the guarantee

```
at-least-once:  poll -> PROCESS -> commit offset   (crash before commit -> reprocess)
at-most-once:   poll -> commit offset -> PROCESS    (crash after commit -> lost)
```

Interview Q / Follow-ups

- Difference between the three guarantees and how to configure each.
- Is exactly-once “real”? *(within Kafka via txns; end-to-end needs idempotency.)*
- What do `acks=all` + `min.insync.replicas` protect against?
- Why is at-least-once the pragmatic default?

8.4 Retry & Dead Letter Queue (DLQ)

Core Concept

Transient failures → **retry with backoff**; poison messages (always fail) → route to a **Dead Letter Queue/Topic** after N attempts so the pipeline isn’t blocked. In Spring Kafka: `DefaultErrorHandler` + `DeadLetterPublishingRecoverer` (retryable vs non-retryable exceptions, backoff). RabbitMQ: DLX (dead-letter exchange) + TTL for delayed retry.

ASCII

```
consume -> fail -> retry (backoff) x N -> still failing -> DLQ (orders.DLT)
DLQ monitored/alerted -> manual or automated reprocessing
```

Common Mistakes / Best Practices

Blocking retries on the main consumer (stalls the partition) — prefer non-blocking retry topics; infinite retries on poison messages; no DLQ monitoring. Always attach metadata (original topic, exception, attempts) to DLQ records.

Interview Q

What is a DLQ; when does a message go there; blocking vs non-blocking retries; how to reprocess a DLQ.

8.5 Consumer Lag & Offsets in Practice

- **Consumer lag** = latest offset − committed offset (how far behind). High lag = consumers can't keep up → scale consumers (up to #partitions), optimize processing, or increase partitions.
- **Offset commit**: auto (`enable.auto.commit`, risky) vs manual (`ack-mode=MANUAL`, precise). Commit after successful processing for at-least-once.

Interview Q

How to diagnose Kafka lag; how to scale consumers; auto vs manual commit.

8.6 RabbitMQ concepts (contrast)

- **Exchange** types: **direct** (routing key exact), **topic** (wildcard patterns), **fanout** (broadcast), **headers**.
- **Queue** bound to exchange; consumers ack (`basic.ack`/nack); unacked messages redelivered.
- **Prefetch (QoS)** limits unacked messages per consumer (backpressure).
- **DLX** + message TTL for retry/delay; priority queues.

Interview Q

Exchange types and routing; ack/nack/requeue; prefetch and why it matters.

Module 8 — One-Page Cheat Sheet

Topic	Key point
Kafka vs Rabbit	log+replay+throughput vs broker+routing+per-msg semantics
Partition	ordering + parallelism unit; key → same partition
Consumer group	1 partition ↔ 1 consumer in group; other groups fan-out
Offset	consumer progress; commit timing sets guarantee
At-least-once	commit after process; needs idempotent consumer (default)
At-most-once	commit before process; may lose
Exactly-once	idempotent+transactional producer; end-to-end needs idempotency
acks=all + ISR	durability, no loss on broker failure
DLQ	poison messages after N retries; monitor + reprocess
Lag	latest−committed; scale up to #partitions
Rabbit exchanges	direct/topic/fanout/headers; prefetch backpressure

Module 8 — Top Interview Questions

1. Kafka vs RabbitMQ — when each?
2. How does Kafka guarantee ordering; its limits?
3. Explain consumer groups, partitions, offsets.
4. At-most vs at-least vs exactly-once; how to configure.
5. Is exactly-once real end-to-end? (idempotency)
6. What is a DLQ; blocking vs non-blocking retry.
7. `acks` and `min.insync.replicas`.
8. How to diagnose and fix consumer lag.
9. Auto vs manual offset commit.
10. RabbitMQ exchange types and routing.

Module 8 — Common Mistakes

- Assuming global ordering across partitions.
- Auto-commit causing lost/duplicate processing.
- No idempotency with at-least-once delivery.
- Blocking retries stalling a partition; no DLQ.
- More consumers than partitions (idle consumers).

Module 8 — Mock Interview

1. *“Guarantee per-customer event ordering at scale.”* → key by customerId → same partition; don’t rely on global order.
2. *“A consumer keeps crashing on one bad message and blocks everything.”* → route to DLQ after N retries (non-blocking retry topics).
3. *“How do you avoid double-charging on Kafka retries?”* → at-least-once + idempotent consumer (dedup on eventId), and/or transactional EOS.
4. *“Consumer lag is growing.”* → scale consumers up to #partitions, add partitions, speed up processing, batch.

5. *“Choose Kafka or RabbitMQ for a task queue with priorities and per-message TTL.”* → RabbitMQ (priority queues, TTL, flexible routing).

Next → Module 9: Redis.

Module 9 — Redis

> Redis is the default answer for caching, distributed locks, session stores, and rate limiting. Interviewers want caching strategies, invalidation, and the correctness pitfalls of distributed locks.

9.1 Caching Fundamentals & Strategies

Why Interviewers Ask This

Caching is the highest-leverage performance tool and the source of the two hardest problems in CS ("cache invalidation and naming things"). They test whether you pick the right strategy and handle staleness/failures.

Core Concept

Redis is an in-memory key-value store (sub-ms latency). Cache = keep hot data close to compute to avoid slow backends.

Strategies

- **Cache-Aside (Lazy loading)** — app checks cache; miss → load from DB → populate cache. Most common. App owns cache logic. Risk: first-request miss latency, stale data until TTL/eviction.
- **Read-Through** — cache library loads from DB on miss transparently.
- **Write-Through** — write to cache **and** DB synchronously → cache always fresh, higher write latency.
- **Write-Behind (Write-Back)** — write to cache, async flush to DB → fast writes, risk of loss on crash.
- **Write-Around** — write to DB only; cache populated on later reads (good for write-heavy, rarely-read data).

ASCII — Cache-Aside

```
 read:  app -> cache? HIT -> return
                    MISS -> DB -> put in cache (TTL) -> return
 write: app -> DB -> INVALIDATE (delete) cache key
```

Real Production Example (Spring)

```
@Cacheable(value="products", key="#id")
Product get(Long id) { return repo.findById(id).orElseThrow(); }

@CacheEvict(value="products", key="#p.id")
Product update(Product p) { return repo.save(p); }
```

Trade-offs / Common Mistakes

- **Invalidation**: prefer **delete on write** over update-in-place (avoids race where a stale write wins). "Cache-aside + evict" is the safe default.
- **Stampede / thundering herd**: many misses hit the DB at once when a hot key expires → use locks/`@Cacheable` sync, request coalescing, or jittered TTL.
- **Cache penetration**: repeated misses for non-existent keys → cache negative results (short TTL) or bloom filter.
- **Cache avalanche**: many keys expire simultaneously → randomize/jitter TTLs.
- Caching without TTL; caching user-specific data under a shared key.

Interview Q / Follow-ups

- Cache-aside vs write-through vs write-behind — trade-offs.
- How do you invalidate cache safely on writes? *(delete, not update.)*
- Cache stampede/penetration/avalanche — what are they and mitigations?

- How to keep cache consistent with the DB? *(TTL + evict-on-write; accept eventual consistency; or write-through.)*

9.2 TTL & Eviction

- **TTL** (`EXPIRE`) bounds staleness and memory. Always set one for caches.
- **maxmemory-policy** eviction: `allkeys-lru`, `allkeys-lfu`, `volatile-ttl`, `noeviction`. Pick LRU/LFU for caches; `noeviction` for durable stores (writes fail when full).

Interview Q

Why set TTL; which eviction policy for a pure cache; LRU vs LFU.

9.3 Distributed Lock

Why Interviewers Ask This

Distributed locks are subtly incorrect if done naively — a favorite trap.

Core Concept

Coordinate mutually exclusive work across instances (e.g. run a scheduled job once). Basic: `SET key uuid NX PX 30000` (set if not exists + expiry). Release **only if you own it** via a Lua compare-and-delete (never a plain `DEL` — you might delete someone else's lock after your TTL expired).

```
if redis.call('get', KEYS[1]) == ARGV[1]
then return redis.call('del', KEYS[1]) else return 0 end
```

Pitfalls / Trade-offs

- **TTL too short** → lock expires mid-work → two holders. **Too long** → slow recovery on crash.
- Single-node Redis lock isn't safe under failover; **Redlock** (multi-node) is debated (Kleppmann's critique) — for strict correctness use a fencing token or a real consensus system (ZooKeeper/etcd). For "best-effort mutual exclusion," single-node + fencing is usually fine.
- Use **Redisson** (`RLock`) in Java — handles renewal (watchdog) and safe release.

ASCII

```
SET lock:job <uuid> NX PX 30000  -> acquired? do work
work must finish < TTL (or renew via watchdog)
release: Lua { if value==uuid then DEL }
```

Interview Q / Follow-ups

- How to implement a distributed lock correctly (NX+PX+owner check)?
- Why is `DEL` on release dangerous?
- What is Redlock and its criticism? What are fencing tokens?

9.4 Session Store

Externalize HTTP sessions to Redis (Spring Session) so any instance can serve any request → stateless app tier, no sticky sessions, survives restarts. Alternative to JWT for stateful web apps; enables central logout/revocation.

Interview Q

Session-in-Redis vs JWT; why externalize sessions (horizontal scaling).

9.5 Rate Limiter

Core Concept

Protect APIs from abuse/overload. Common algorithms: - **Fixed window** — count per window; simple but bursty at boundaries. - **Sliding window (log/counter)** — smoother, more accurate. - **Token bucket** — allow bursts up to bucket size, refill at rate (most common; Bucket4j/Redis). - **Leaky bucket** — constant outflow.

Redis implements these atomically (INCR+EXPIRE, sorted sets, or Lua) so limits are shared across instances.

ASCII — Token Bucket

```
tokens refill at R/sec up to capacity C
request: tokens>=1 ? consume 1, allow : reject (429)
```

Interview Q / Follow-ups

- Fixed vs sliding window vs token bucket — trade-offs.
- Why do it in Redis (shared, atomic) rather than per-instance?
- How to return 429 with `Retry-After`?

Module 9 — One-Page Cheat Sheet

Topic	Key point
Cache-aside	app-managed; miss→DB→populate; evict on write
Write-through	sync to cache+DB; fresh, slower writes
Write-behind	async to DB; fast, risk loss
Invalidation	delete key on write (not update)
Stampede	lock/coalesce + jittered TTL
Penetration	cache negatives / bloom filter
Avalanche	randomize TTLs
TTL/eviction	always TTL; allkeys-lru/lfu for caches
Distributed lock	SET NX PX + owner-check Lua release; Redisson watchdog
Session store	Spring Session → stateless app tier
Rate limit	token bucket in Redis (atomic, shared)

Module 9 — Top Interview Questions

1. Caching strategies and when to use each.
2. How do you keep cache consistent with the DB / invalidate safely?
3. Cache stampede/penetration/avalanche and mitigations.
4. Implement a correct distributed lock; why is naive release unsafe?
5. Redlock and its criticism; fencing tokens.
6. Session in Redis vs JWT.
7. Rate limiting algorithms; why Redis.
8. Eviction policies; LRU vs LFU.
9. Why always set a TTL?

10. `@Cacheable` / `@CacheEvict` semantics in Spring.

Module 9 — Common Mistakes

- No TTL → stale/unbounded memory.
- Updating cache in place instead of evicting → stale races.
- `DEL` release without owner check → deleting others' locks.
- Caching per-user data under a shared key.
- Relying on single-node Redis lock for strict correctness.

Module 9 — Mock Interview

1. *"DB is hammered when a hot cache key expires."* → cache stampede; lock/coalesce loads, `@Cacheable(sync=true)`, jitter TTL.
2. *"Run a nightly job exactly once across 5 instances."* → distributed lock (`SET NX PX`) or Redisson `RLock`; ensure TTL > job time or use watchdog.
3. *"Keep the product cache consistent after updates."* → evict key on write, short TTL, accept eventual consistency (or write-through).
4. *"Rate-limit an API to 100 req/min/user across instances."* → token bucket in Redis (atomic INCR/Lua), 429 + Retry-After.
5. *"Attackers query random non-existent IDs, bypassing cache."* → cache penetration; cache negative results with short TTL / bloom filter.

Next → Module 10: Database.

Module 10 — Database

> Interview-relevant DB fundamentals for backend engineers: ACID, isolation levels & anomalies, indexing, query optimization, connection pooling (HikariCP), and schema migrations (Flyway/Liquibase).

10.1 Transactions & ACID

Core Concept

A transaction is an atomic unit of work. **ACID**: - **Atomicity** — all or nothing (rollback on failure). - **Consistency** — moves DB from one valid state to another (constraints hold). - **Isolation** — concurrent txns don't corrupt each other (degree set by isolation level). - **Durability** — committed data survives crashes (WAL/redo log, fsync).

Interview Q

Explain each ACID property with an example (e.g. bank transfer = atomicity + consistency). How is durability implemented? *(write-ahead log + fsync.)*

10.2 Isolation Levels & Anomalies

Why Interviewers Ask This

This is the crux of concurrency correctness and directly ties to `@Transactional(isolation=...)`.

Anomalies

- **Dirty read** — read uncommitted data.
- **Non-repeatable read** — re-reading a row returns different data (another txn updated + committed).
- **Phantom read** — re-running a range query returns new rows.
- **Lost update** — two txns overwrite each other's update.

Levels (SQL standard) — each prevents more

Level	Dirty	Non-repeatable	Phantom
READ UNCOMMITTED	possible		
READ COMMITTED	✗		
REPEATABLE READ	✗	✗	(SQL std; MySQL InnoDB prevents via gap locks)
SERIALIZABLE	✗	✗	✗

Defaults: **PostgreSQL & Oracle = READ COMMITTED**; **MySQL InnoDB = REPEATABLE READ**. Higher isolation = more locking/aborts = less concurrency. Many DBs use **MVCC** (readers don't block writers) + snapshot isolation.

ASCII

```
lower isolation  ---- more concurrency, more anomalies ---->
READ UNCOMMITTED < READ COMMITTED < REPEATABLE READ < SERIALIZABLE
<---- fewer anomalies, more locking/contention -----
```

Interview Q / Follow-ups

- Define the anomalies; which level prevents which.
- Default isolation in Postgres vs MySQL.

- What is MVCC / snapshot isolation?
- How does isolation relate to optimistic/pessimistic locking (Module 5)?

10.3 Indexes

Core Concept

An index is a sorted data structure (usually a **B+ tree**) that speeds lookups from O(n) scan to O(log n), at the cost of extra storage and slower writes (index maintenance).

Types & concepts

- **B-tree** — range + equality (default).
- **Hash** — equality only.
- **Composite** — multi-column; order matters (**leftmost-prefix rule**: an index on `(a,b,c)` helps `a`, `a,b`, `a,b,c`, not `b` alone).
- **Covering index** — index includes all columns a query needs → no table lookup.
- **Clustered** (table stored in index order, e.g. PK in InnoDB) vs **non-clustered/secondary** (points to row).
- **Selectivity** — high-cardinality columns index well; low-cardinality (e.g. boolean) often doesn't.

Common Mistakes

Indexing everything (write penalty, storage); function on indexed column (`WHERE lower(email)=` breaks index unless functional index); wrong composite order; not indexing FKs/join columns; ignoring `LIKE '%x'` (can't use B-tree).

Interview Q / Follow-ups

- How does a B+ tree index speed queries?
- Leftmost-prefix rule; composite index ordering.
- What is a covering index? Clustered vs non-clustered.
- When does an index hurt / not get used?

10.4 Query Optimization

Techniques

- **`EXPLAIN`/`EXPLAIN ANALYZE`** — read the plan: seq scan vs index scan, join type (nested loop/hash/merge), rows estimated vs actual, sort/filter.
- Add/fix indexes; rewrite to be **SARGable** (avoid functions on columns).
- Avoid `SELECT *`; select needed columns (enable covering indexes).
- Fix N+1 (Module 5). Paginate (keyset for deep pages).
- Batch writes; avoid per-row round trips.
- Watch for implicit type casts breaking index usage.
- Update statistics (`ANALYZE`) so the planner estimates well.

ASCII — reading a plan

```
EXPLAIN ANALYZE SELECT ...
  -> Seq Scan on orders (cost.. rows=1M)   <-- red flag: full scan
  after index: Index Scan using idx_orders_user (rows=25)  <-- good
```

Interview Q

- How do you find and fix a slow query? *(EXPLAIN → index/rewrite → verify.)*

- What makes a predicate non-SARGable?
- Nested-loop vs hash vs merge join — when each?

10.5 Connection Pooling (HikariCP)

Why Interviewers Ask This

Connection pool exhaustion is a top production incident (Module 13). HikariCP is Spring Boot's default.

Core Concept

Opening a DB connection is expensive (TCP + auth). A **pool** keeps a bounded set of open connections and hands them out, blocking (up to `connectionTimeout`) when none are free.

Key settings

- `maximumPoolSize` — the cap (right-sizing is critical; often **fewer** than you think — see PostgreSQL's formula `connections ≈ ((core_count*2)+effective_spindles)`).
- `minimumIdle`, `connectionTimeout` (default 30s), `idleTimeout`, `maxLifetime` (< DB/infra idle timeout), `leakDetectionThreshold`.

Common Mistakes / Debugging

- Pool too small → threads wait/`connectionTimeout` → latency spikes/timeouts.
- Pool too large → overwhelms DB (context switching, memory), worse throughput.
- **Holding connections during remote calls / long transactions** → exhaustion.
- Symptoms: `Connection is not available, request timed out`; monitor active vs idle, pending threads (Micrometer/HikariCP metrics).

Interview Q / Follow-ups

- Why use a connection pool?
- How do you size `maximumPoolSize`? Bigger isn't better — why?
- What causes pool exhaustion and how to diagnose it?
- `maxLifetime` vs DB idle timeout — why align them?

10.6 Schema Migrations: Flyway vs Liquibase

	Flyway	Liquibase
Format	versioned SQL (`V1__init.sql`)	XML/YAML/JSON/SQL changelogs
Style	SQL-first, simple	DB-agnostic abstraction
Rollback	forward-fix (undo in paid)	built-in rollback tags
Best for	teams comfortable with SQL	multi-DB, declarative

Both track applied migrations in a metadata table and run pending ones at startup; integrate with Spring Boot automatically.

Best Practices

Migrations are **immutable + forward-only** (never edit an applied script); backwards-compatible for zero-downtime deploys (expand/contract: add column → deploy → backfill → switch → drop later); separate DDL from data migration; test on a copy.

Interview Q

- Flyway vs Liquibase.
- How do you do a zero-downtime schema change? *(expand/contract, backward-compatible steps.)*
- Why never modify an applied migration?

Module 10 — One-Page Cheat Sheet

Topic	Key point
ACID	atomicity, consistency, isolation, durability (WAL)
Anomalies	dirty / non-repeatable / phantom / lost update
Isolation	RU<RC<RR<SER; Postgres=RC, MySQL=RR; MVCC snapshots
Index	B+ tree O(log n); leftmost-prefix; covering; clustered vs secondary
SARGable	no functions on indexed columns
EXPLAIN	seq scan bad; check estimates vs actual, join type
HikariCP	bounded pool; small is often better; align maxLifetime
Pool exhaustion	long tx / remote calls holding connections
Migrations	Flyway (SQL) vs Liquibase; immutable, forward-only, expand/contract

Module 10 — Top Interview Questions

1. Explain ACID; how is durability achieved?
2. Isolation levels and the anomalies each prevents; defaults per DB.
3. What is MVCC?
4. How do indexes work; leftmost-prefix; covering index.
5. When is an index not used / harmful?
6. How do you optimize a slow query (EXPLAIN)?
7. Why connection pooling; how to size HikariCP.
8. Causes and diagnosis of pool exhaustion.
9. Flyway vs Liquibase; zero-downtime migrations.
10. Clustered vs non-clustered index.

Module 10 — Common Mistakes

- Over-indexing (write cost) / not indexing join & FK columns.
- Functions on indexed columns (non-SARGable).
- Oversized connection pool overwhelming the DB.
- Long transactions holding connections.
- Editing an already-applied migration.

Module 10 — Mock Interview

1. *“An API got slow after data grew.”* → `EXPLAIN ANALYZE`; likely seq scan → add proper (composite/covering) index; verify plan.

2. *“`Connection is not available` errors under load.”* → pool exhaustion; check for long tx / remote calls inside tx, size the pool, add timeouts.
3. *“Two transfers double-spend a balance.”* → lost update; raise isolation or use optimistic/pessimistic locking.
4. *“Add a NOT NULL column to a 500M-row table with zero downtime.”* → expand/contract: add nullable + default, backfill in batches, then enforce constraint.
5. *“Why is MySQL’s default isolation different from Postgres?”* → InnoDB REPEATABLE READ (gap locks prevent phantoms) vs Postgres READ COMMITTED; both use MVCC.

Next → Module 11: Observability.

Module 11 — Observability

> “You can’t fix what you can’t see.” Observability = **metrics + logs + traces** (the three pillars). Interviewers want the Spring stack (Actuator + Micrometer + Prometheus + Grafana) and how you trace a request across microservices.

11.1 The Three Pillars

Pillar	Answers	Tooling
Metrics	how much / how fast (aggregates)	Micrometer → Prometheus → Grafana
Logs	what happened (discrete events)	SLF4J/Logback → ELK/Loki
Traces	where time went across services	Micrometer Tracing / OpenTelemetry → Jaeger/Tempo/Zipkin

Monitoring = watching known metrics/alerts; **observability** = ability to ask new questions about unknown failure modes from your telemetry.

11.2 Actuator & Micrometer

Actuator

Exposes operational endpoints (Module 3.9): `/actuator/health` (probes), `/metrics`, `/prometheus`, `/loggers`, `/threaddump`, `/heapdump`, `/httpexchanges`.

Micrometer

A **vendor-neutral metrics facade** (“SLF4J for metrics”) built into Spring Boot. You instrument once; export to Prometheus, Datadog, CloudWatch, etc.

Meter types

- **Counter** — monotonically increasing (requests, errors).
- **Gauge** — current value (queue size, cache entries, pool active).
- **Timer** — count + total time + percentiles (latency).
- **DistributionSummary** — distributions (payload sizes).
- **Tags/labels** — dimensions (`uri`, `status`, `method`) — keep cardinality low!

Real Production Example

```
Timer.Sample s = Timer.start(registry);
try { doWork(); }
finally { s.stop(registry.timer("order.process", "type", type)); }

meterRegistry.counter("orders.created", "channel", channel).increment();
```

Boot auto-instruments HTTP (`http.server.requests`), JVM (memory/GC/threads), HikariCP, Kafka, etc.

Common Mistakes

High-cardinality tags (user id, request id in labels) blow up Prometheus memory/series. Use bounded label values; put unbounded ids in traces/logs.

Interview Q

- Micrometer purpose; counter vs gauge vs timer.
- Why is metric cardinality dangerous?
- What does `http.server.requests` give you (RED metrics)?

11.3 Prometheus & Grafana

- **Prometheus** — time-series DB that **scrapes** `/actuator/prometheus` periodically; stores metrics; **PromQL** for queries; **Alertmanager** for alerts.
- **Grafana** — dashboards/visualization over Prometheus (and logs/traces).
- **RED method** (request-rate, errors, duration) for services; **USE** (utilization, saturation, errors) for resources.

ASCII

```
app /actuator/prometheus  <--scrape-- Prometheus --query--> Grafana dashboards
                                          |
                                    Alertmanager -> Slack/PagerDuty
```

Interview Q

Pull (Prometheus scrape) vs push model; what to alert on (SLOs: latency, error rate, saturation).

11.4 Centralized Logging

Core Concept

Aggregate logs from all instances/services into one searchable store (ELK: Elasticsearch+Logstash+Kibana, or Loki+Grafana). Use **structured JSON logs** + **correlation/trace id in MDC** so you can follow one request across services.

Best Practices

- JSON encoder (logback) with fields: timestamp, level, logger, traceId, spanId, service, message.
- Put `traceId` in MDC via a filter; propagate downstream (headers).
- Log levels appropriate; never log secrets/PII; sample noisy logs.

Interview Q

Why structured logging; how to correlate logs across microservices (trace id in MDC); log vs metric vs trace — when to use which.

11.5 Distributed Tracing & OpenTelemetry

Why Interviewers Ask This

In microservices, one user request fans out to many services; tracing shows the end-to-end path and where latency is spent.

Core Concept

- A **trace** = one request's journey; composed of **spans** (units of work). Each span has a `traceId` (shared), `spanId`, parent span id, timing, tags.
- **Context propagation**: the trace context travels via headers (W3C `traceparent`) across HTTP/Kafka so spans link up.

- **Micrometer Tracing** (successor to Spring Cloud Sleuth) + **OpenTelemetry** (vendor-neutral standard for traces/metrics/logs) export to Jaeger/Tempo/Zipkin.

ASCII — a trace

```
traceId=abc
┌ gateway  [span 1] ──────────────────────────── 120ms
│  ├ order-service [span 2] ──────────── 90ms
│  │   ├ DB query [span 3] ── 20ms
│  │   └ payment-service [span 4] ── 55ms  <-- bottleneck
│  └ ...
```

Real Production Example

p99 latency spikes on checkout. The trace shows most time in `payment-service`'s external call → add timeout/circuit breaker there and cache pricing.

Interview Q / Follow-ups

- What are traces and spans; how is context propagated (traceparent)?
- Sleuth vs Micrometer Tracing vs OpenTelemetry.
- How would you find which service causes a latency spike? *(distributed trace.)*
- Sampling — why (cost) and head vs tail sampling.

Hands-on Exercise

Add Micrometer Tracing + OTel exporter; make two services call each other; confirm one trace with linked spans and a propagated `traceId` in logs.

Module 11 — One-Page Cheat Sheet

Topic	Key point
Pillars	metrics (aggregate), logs (events), traces (cross-service path)
Actuator	health/metrics/prometheus/loggers/threaddump
Micrometer	metrics facade; counter/gauge/timer; low cardinality!
Prometheus	scrapes /prometheus; PromQL; Alertmanager
Grafana	dashboards; RED (rate/errors/duration), USE
Logging	structured JSON + traceId in MDC; never log secrets
Tracing	trace=spans; W3C traceparent propagation; OTel standard
Probes	liveness vs readiness via health groups

Module 11 — Top Interview Questions

1. Three pillars of observability; when use each.
2. What is Micrometer; meter types; cardinality pitfall.
3. How does Prometheus collect metrics (pull/scrape)?
4. How do you correlate logs across microservices?
5. Traces vs spans; context propagation.
6. Sleuth vs Micrometer Tracing vs OpenTelemetry.
7. How to find a latency bottleneck across services.
8. What to alert on (SLOs); RED/USE.
9. Liveness vs readiness probes.

10. Monitoring vs observability.

Module 11 — Common Mistakes

- High-cardinality metric labels (ids) → Prometheus OOM.
- Unstructured logs; no trace id.
- Logging secrets/PII.
- Alerting on causes instead of symptoms (SLOs).
- No sampling → tracing cost explosion.

Module 11 — Mock Interview

1. *“Checkout p99 spiked; where do you look?”* → distributed trace to find the slow span/service; correlate with metrics; check that service’s dependencies.
2. *“How do you follow one request across 6 services in logs?”* → propagate a traceId (W3C traceparent) into MDC; structured JSON logs; search by traceId.
3. *“Prometheus is OOMing.”* → high-cardinality labels (user/request ids); remove them, keep dimensions bounded.
4. *“What alerts would you set for a service?”* → SLO-based: error rate, p99 latency, saturation (RED/USE), not raw CPU alone.
5. *“Liveness vs readiness probe?”* → liveness = restart if dead; readiness = remove from LB until ready/deps healthy.

Next → Module 12: Docker & Kubernetes.

Module 12 — Docker & Kubernetes

> Interview-relevant containerization + orchestration for a Spring Boot service. Focus: efficient images, the core K8s objects, config/secrets, probes, and how a Spring Boot app is deployed and scaled.

12.1 Docker & Dockerfile

Core Concept

A **container** packages an app + its dependencies into an isolated, portable unit sharing the host kernel (namespaces + cgroups) — lighter than a VM (no guest OS). An **image** is the immutable, layered template; a **container** is a running instance.

Efficient Spring Boot Dockerfile (multi-stage)

```
# ---- build ----
FROM eclipse-temurin:21-jdk AS build
WORKDIR /app
COPY . .
RUN ./mvnw -q -DskipTests package

# ---- run ----
FROM eclipse-temurin:21-jre
WORKDIR /app
COPY --from=build /app/target/app.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java","-XX:MaxRAMPercentage=75","-jar","app.jar"]
```

Best practices / layering

- **Multi-stage build** → small runtime image (JRE, not JDK).
- **Layer caching**: copy `pom.xml`/deps and resolve **before** copying source, so code changes don't re-download dependencies. Spring Boot **layered jars** / `bootBuildImage` (buildpacks) split deps vs app for better caching.
- Run as **non-root** user; pin base image; `.dockerignore`; small base (distroless/alpine caveats with glibc).
- Set memory-aware flags (`MaxRAMPercentage`) so the JVM respects cgroup limits.

ASCII — image layers

```
 [ base JRE layer ] (rarely changes, cached)
 [ dependencies layer ] (changes when pom changes)
 [ application classes ] (changes every build)
 -> only top layers rebuild/push
```

Interview Q / Follow-ups

- Container vs VM.
- Image vs container; what are layers; how does caching work.
- Why multi-stage builds; JDK vs JRE in the final image.
- How do you make the JVM respect container memory limits?

12.2 Docker Compose

Declaratively run multi-container local stacks (app + Postgres + Redis + Kafka) with one `docker compose up`. Defines services, networks, volumes, env, depends_on. Great for local dev/integration tests (Testcontainers uses similar ideas), **not** production orchestration.

Interview Q

When Compose vs Kubernetes; how services discover each other in Compose (service name DNS).

12.3 Kubernetes Core Objects

Why Interviewers Ask This

K8s is the de-facto production runtime. They want the core objects and how a Spring Boot app is deployed, configured, scaled, and exposed.

Objects

- **Pod** — smallest deployable unit; one or more containers sharing network/ storage; ephemeral (gets a new IP on reschedule).
- **ReplicaSet** — keeps N pod replicas running (usually managed by a Deployment).
- **Deployment** — declarative pods + rolling updates/rollbacks; the workload you usually create.
- **Service** — stable virtual IP + DNS load-balancing across pods. Types: **ClusterIP** (internal), **NodePort**, **LoadBalancer** (cloud LB).
- **Ingress** — HTTP(S) routing (host/path) + TLS into Services (needs an ingress controller, e.g. nginx).
- **ConfigMap** — non-secret config (env vars / mounted files).
- **Secret** — sensitive data (base64-encoded, not encrypted by default; enable encryption-at-rest / external secret managers).
- Also: **HPA** (Horizontal Pod Autoscaler by CPU/metrics), **StatefulSet** (stable identity/storage for stateful apps), **DaemonSet**, **Job/CronJob**, **Namespace**, **PV/PVC**.

ASCII — request path

```
Internet -> Ingress (host/path, TLS)
         -> Service (ClusterIP, DNS, LB across pods)
         -> Pod(s)  [ Spring Boot container ]
ConfigMap/Secret -> env/volume -> app
HPA watches CPU/metrics -> scales Deployment replicas
```

Deployment lifecycle / probes

- **Rolling update**: new ReplicaSet scaled up while old scaled down (maxSurge/ maxUnavailable); readiness gates traffic; rollback with `kubectl rollout undo`.
- **Probes** (map to Actuator health groups):
- **liveness** → restart container if it hangs (`/actuator/health/liveness`).
- **readiness** → remove from Service until ready/deps ok (`/actuator/health/readiness`).
- **startup** → protect slow-starting JVMs from premature liveness kills.

Real Production Example

Spring Boot Deployment with 3 replicas, resource requests/limits, `ConfigMap` for `application.yml`, `Secret` for DB creds, readiness+liveness probes on Actuator, HPA scaling on CPU 70%, exposed via ClusterIP Service + Ingress with TLS.

Common Mistakes

- No resource requests/limits → noisy-neighbor / OOMKilled / scheduling issues.
- Liveness probe too aggressive → restart loops during slow startup (use startup probe).
- Storing secrets in ConfigMaps / images.
- Assuming pod IP/storage is stable (it isn't — use Services/PVCs).
- JVM not container-aware (old JDKs) → OOMKilled.

Interview Q / Follow-ups

- Pod vs Deployment vs Service vs Ingress.
- Service types (ClusterIP/NodePort/LoadBalancer).
- ConfigMap vs Secret; are Secrets encrypted?
- Liveness vs readiness vs startup probes.
- How does a rolling update / rollback work?
- How does HPA autoscale? How do pods find each other? *(Service DNS.)*

Hands-on Exercise

Write a Deployment + Service + Ingress + ConfigMap + Secret for a Spring Boot app; add liveness/readiness probes on Actuator; add an HPA; do a rolling update and a rollback.

Module 12 — One-Page Cheat Sheet

Object	Role
Pod	smallest unit; ephemeral; shares net/storage
ReplicaSet	maintains N replicas
Deployment	declarative pods + rolling update/rollback
Service	stable IP/DNS, LB across pods (ClusterIP/NodePort/LB)
Ingress	HTTP(S) routing + TLS
ConfigMap	non-secret config
Secret	sensitive data (base64; encrypt at rest)
HPA	autoscale replicas by metrics
Probes	liveness (restart), readiness (LB gate), startup (slow boot)

Docker: multi-stage, JRE runtime, layer caching, non-root, `MaxRAMPercentage`.

Module 12 — Top Interview Questions

1. Container vs VM; image vs container; layers & caching.
2. Why multi-stage builds; JDK vs JRE final image.
3. Pod vs Deployment vs ReplicaSet vs Service vs Ingress.
4. Service types and when to use each.
5. ConfigMap vs Secret; Secret security.
6. Liveness vs readiness vs startup probes (+ Actuator mapping).
7. Rolling update and rollback mechanics.
8. HPA autoscaling; requests vs limits.
9. Make the JVM respect container memory.
10. How do pods discover services?

Module 12 — Common Mistakes

- Shipping JDK / fat images; no `.dockerignore`.
- Missing resource requests/limits → OOMKilled/eviction.
- Aggressive liveness probe → restart loops.
- Secrets in images/ConfigMaps.
- Assuming stable pod IP/storage.

Module 12 — Mock Interview

1. *"Your pod keeps restarting right after deploy."* → liveness probe firing during slow JVM startup; add a startup probe / raise initialDelay.
2. *"Optimize a 700MB Spring image."* → multi-stage, JRE base, layered jar, `.dockerignore`, distroless.
3. *"How does config differ per environment in K8s?"* → ConfigMaps/Secrets per namespace + profiles; mount or env-inject.
4. *"How does zero-downtime deploy work?"* → rolling update gated by readiness probe; rollback via `rollout undo`.
5. *"App gets OOMKilled though heap looks fine."* → total container memory (heap + metaspace + threads + native) exceeds the limit; set `MaxRAMPercentage` and right-size limits.

Next → Module 13: Production Scenarios (debugging).

Module 13 — Production Scenarios (Debugging)

> Senior interviews lean heavily on “walk me through debugging X in production.” The winning approach is always: **symptom → hypotheses → collect evidence (metrics/logs/traces/dumps) → isolate → fix → verify → prevent.** Below is a playbook per incident with the exact tools.

13.1 The universal debugging framework

```
1. Define the symptom & blast radius (which endpoint/service, since when, % affected)
2. Correlate: dashboards (RED/USE), recent deploys/config changes, traces
3. Form hypotheses (resource? dependency? code? data?)
4. Collect evidence: thread dump, heap dump, GC log, DB stats, slow-query log
5. Isolate the bottleneck (one variable at a time)
6. Mitigate (rollback/scale/flag) then fix root cause
7. Verify with metrics; add alert/test to prevent recurrence
```

Core JVM tools: `jstack` (threads), `jmap` / `jcmd GC.heap_dump` (heap), `jstat` /GC logs (GC), `jcmd`, async-profiler / JFR (CPU + locks + allocation), Actuator (`/threaddump`, `/heapdump`, `/metrics`).

13.2 Memory Leak

- **Symptom:** heap grows over time, Old Gen never shrinks after full GC, eventually `OutOfMemoryError: Java heap space`; rising GC time.
- **Diagnose:** enable `-XX:+HeapDumpOnOutOfMemoryError`; take heap dump (`jmap -dump:live` / Actuator `/heapdump`); open in Eclipse **MAT** → dominator tree + retained size → find the growing object graph and GC roots.
- **Common causes:** unbounded static/`ThreadLocal` collections/caches (no TTL/max size), listeners/connections never removed, classloader leaks (Metaspace), caching entities forever, `ThreadLocal` not cleared in pooled threads.
- **Fix:** bound caches (size+TTL), remove listeners, clear `ThreadLocal`, weak references where appropriate.

13.3 High CPU

- **Symptom:** CPU pegged, latency up.
- **Diagnose:** `top -H -p <pid>` to find hot threads → convert TID to hex → match in `jstack`; or **async-profiler**/ JFR flame graph.
- **Common causes:** infinite/busy loops, excessive GC (see GC log — CPU burned in GC), inefficient algorithm ($O(n^2)$), regex catastrophic backtracking, serialization, chatty logging.
- **Fix:** optimize hot path, tune GC/allocation, add caching, fix the loop.

13.4 Slow APIs

- **Symptom:** high p95/p99 latency.
- **Diagnose:** distributed **trace** to find the slow span (DB? downstream? external?), DB slow-query log + `EXPLAIN`, check GC pauses, pool waits.
- **Common causes:** N+1 queries, missing index, slow/blocking downstream call without timeout, lock contention, connection-pool wait, large payloads/no pagination, cold cache.
- **Fix:** index/rewrite query, fix N+1 (EntityGraph), add cache, timeouts + circuit breaker, paginate, async/parallelize.

13.5 Deadlocks

- **Symptom:** threads hang, throughput drops, no progress.
- **Diagnose:** `jstack` → “Found one Java-level deadlock” section (which threads hold/wait which locks). DB deadlocks: DB logs / `SHOW ENGINE INNODB STATUS`.
- **Common causes:** inconsistent lock ordering, nested `synchronized`, DB rows locked in different order across transactions.
- **Fix:** consistent global lock ordering, `tryLock` with timeout, reduce lock scope, keep transactions short and same-ordered.

13.6 Connection Pool Exhaustion

- **Symptom:** `HikariPool - Connection is not available, request timed out`; latency spikes; requests queue.
- **Diagnose:** HikariCP metrics (active/idle/pending), thread dump (threads blocked getting a connection), find long-running queries / transactions.
- **Common causes:** long transactions, **remote/HTTP calls inside a transaction**, leaked connections (not closed), pool too small, slow queries holding connections.
- **Fix:** shorten transactions, move I/O outside tx, fix leaks (try-with- resources / Spring-managed), right-size pool, add query timeouts, `leakDetectionThreshold`.

13.7 Kafka Consumer Lag

- **Symptom:** lag (latest−committed offset) grows; data freshness degrades.
- **Diagnose:** `kafka-consumer-groups --describe`, lag metrics; check per- partition lag, consumer CPU, processing time, rebalances.
- **Common causes:** slow processing, too few consumers/partitions, blocking calls per message, frequent rebalances, huge messages.
- **Fix:** scale consumers (≤ #partitions), add partitions, batch processing, async/parallel within consumer, tune `max.poll.records`/`max.poll.interval.ms`, move slow work off the poll thread.

13.8 Redis Cache Misses

- **Symptom:** low hit ratio, DB load high, latency up.
- **Diagnose:** Redis `INFO stats` (keyspace hits/misses), `INFO memory`, evicted keys, monitor TTLs.
- **Common causes:** TTL too short, eviction under memory pressure (LRU/LFU evicting hot keys), cache stampede on expiry, penetration (missing keys), key design (per-user vs shared).
- **Fix:** right-size memory + eviction policy, tune TTL + jitter, stampede protection (lock/coalesce), cache negatives, warm cache.

13.9 Database Bottlenecks

- **Symptom:** DB CPU/IO high, slow queries, lock waits.
- **Diagnose:** slow-query log, `EXPLAIN ANALYZE`, `pg_stat_activity`/ `SHOW PROCESSLIST`, lock/wait stats, index usage stats.
- **Common causes:** missing/wrong indexes, N+1, full scans, hot rows/locks, no pagination, unbatched writes, stale statistics.

- **Fix:** indexes, query rewrite, caching, read replicas, batching, partitioning/ sharding for scale, connection pool tuning.

13.10 Thread Starvation

- **Symptom:** requests queue while CPU idle; pool/executor saturated; `RejectedExecutionException`.
- **Diagnose:** thread dump (many threads BLOCKED/WAITING on I/O or locks), pool metrics (active == max, growing queue).
- **Common causes:** blocking I/O on a small/shared pool (Tomcat workers, `ForkJoinPool.commonPool`), no timeouts, bulkhead missing, deadlock/contention.
- **Fix:** bounded dedicated pools + bulkheads, timeouts, non-blocking/async or virtual threads for I/O, don't block the common pool.

13.11 Circular Dependency (startup)

- **Symptom:** app fails to start; `BeanCurrentlyInCreationException` / "circular reference" (or `Requested bean is currently in creation`).
- **Diagnose:** read the cycle Spring prints (A→B→A).
- **Cause/Fix:** constructor cycle can't be resolved; refactor (extract 3rd bean), `@Lazy` on one dependency, or setter injection (Module 2.6). Note Boot 2.6+ forbids circular refs by default.

13.12 Bean Creation Errors (startup)

- **Symptoms & causes:**
- `NoSuchBeanDefinitionException` — dependency not a bean / not scanned → add stereotype/`@Bean`, check `@ComponentScan` packages.
- `NoUniqueBeanDefinitionException` — multiple candidates → `@Primary`/`@Qualifier`.
- `UnsatisfiedDependencyException` — wrapping the real cause (read the "Caused by" chain — often a property/ DB connection failure).
- `BeanCreationException` — often config/property/DB issues at init.
- **Fix:** read the **root cause chain** bottom-up; verify component scan, profiles active, required properties present, auto-config conditions (`--debug`).

Module 13 — One-Page Cheat Sheet (symptom → tool → cause)

Incident	First tool	Usual root cause
Memory leak	heap dump + MAT	unbounded cache/collection/ThreadLocal
High CPU	top -H + jstack / profiler	busy loop, GC thrash, bad algo
Slow API	distributed trace + EXPLAIN	N+1, missing index, no timeout
Deadlock	jstack	inconsistent lock ordering
Pool exhaustion	Hikari metrics + jstack	tx holding conn during remote call
Kafka lag	consumer-groups –describe	slow processing / too few consumers
Cache miss	Redis INFO stats	TTL/eviction/stampede
DB bottleneck	slow log + EXPLAIN	missing index / N+1 / locks
Thread starvation	thread dump + pool metrics	blocking I/O on small pool
Circular dep	startup log	constructor cycle
Bean errors	root-cause chain	missing/ambiguous bean, bad prop

Module 13 — Top Interview Questions

1. How do you debug a memory leak in production? (heap dump → MAT → dominators)
2. High CPU — how do you find the hot thread? (top -H → jstack/profiler)
3. Latency spike — how do you localize it? (trace → span → EXPLAIN)
4. How do you detect a deadlock? (jstack “Found one Java-level deadlock”)
5. Connection pool exhaustion — causes and fix.
6. Growing Kafka lag — diagnose and remediate.
7. Low cache hit ratio — what do you check?
8. Thread starvation vs deadlock — how to tell apart.
9. App won’t start with a bean error — how do you read it?
10. What data do you always capture before restarting a sick JVM? (thread + heap dump, GC log)

Module 13 — Common Mistakes

- Restarting before capturing dumps (losing evidence).
- Blaming CPU when it’s GC or lock contention.
- Ignoring recent deploys/config changes as the trigger.
- Fixing symptoms (bigger pool) without root cause.

Module 13 — Mock Interview

1. *“Service memory climbs until OOM every few days.”* → likely leak; heap-dump-on-OOM, MAT dominator tree; find unbounded cache; bound size+TTL.
2. *“CPU at 100%, latency up, no traffic spike.”* → `top -H` hot thread → `jstack` → busy loop / GC log shows GC thrash; fix code or allocation.
3. *“Half of* `/checkout` *calls time out at 30s.”* → connection-pool timeout; jstack shows threads waiting for connections; a transaction is calling an external API — move it out, add timeout.
4. *“Kafka lag on one partition only.”* → hot key / uneven partitioning or a slow message; check per-partition processing time; rebalance keys / DLQ poison messages.

5. *“App won’t boot: UnsatisfiedDependencyException.”* → read the Caused-by chain to the bottom — often a missing property or DB connection; fix config, verify profile.

Next → Module 14: Frequently Asked Coding Questions.

Module 14 — Frequently Asked Coding Questions

> Take-home / live-coding staples for Spring Boot backend roles. Each item shows the idiomatic Spring solution and the traps interviewers look for. Keep controllers thin, use DTOs, validate input, handle errors centrally.

14.1 REST API Design (CRUD done right)

```
@RestController
@RequestMapping("/api/v1/orders")
class OrderController {
  private final OrderService service;
  OrderController(OrderService service) { this.service = service; }

  @GetMapping("/{id}")
  OrderDto get(@PathVariable Long id) { return service.get(id); }

  @PostMapping
  ResponseEntity<OrderDto> create(@Valid @RequestBody CreateOrderReq req) {
    OrderDto dto = service.create(req);
    return ResponseEntity.created(URI.create("/api/v1/orders/" + dto.id())).body(dto);
  }

  @PutMapping("/{id}")
  OrderDto update(@PathVariable Long id, @Valid @RequestBody UpdateOrderReq req) {
    return service.update(id, req);
  }

  @DeleteMapping("/{id}")
  @ResponseStatus(HttpStatus.NO_CONTENT)
  void delete(@PathVariable Long id) { service.delete(id); }
}
```

Traps: correct status codes (201 + `Location`, 204 on delete), DTOs not entities, versioned path, idempotent PUT/DELETE.

14.2 Pagination

```
@GetMapping
Page<OrderDto> list(@RequestParam(defaultValue="0") int page,
                    @RequestParam(defaultValue="20") int size,
                    @RequestParam(defaultValue="createdAt,desc") String sort) {
  return service.list(PageRequest.of(page, Math.min(size, 100), parseSort(sort)));
}
```

Traps: cap page size; `Page` (with count) vs `Slice` (no count); **keyset/seek pagination** for deep pages (`WHERE id < :lastId ORDER BY id DESC LIMIT n`) since `OFFSET` degrades on large tables.

14.3 File Upload

```
@PostMapping(value="/files", consumes=MediaType.MULTIPART_FORM_DATA_VALUE)
FileDto upload(@RequestPart("file") MultipartFile file) throws IOException {
  if (file.isEmpty()) throw new BadRequestException("empty file");
  // validate size & content-type; stream to storage (S3) — don't hold in memory
  try (InputStream in = file.getInputStream()) { storage.put(key, in, file.getSize()); }
  return new FileDto(key, file.getOriginalFilename(), file.getSize());
}
```

Config: `spring.servlet.multipart.max-file-size`, `max-request-size`. **Traps:** validate size/type (magic bytes, not just extension), stream (don't load whole file into heap), sanitize filename, virus scan, store in object storage not the DB.

14.4 Exception Handling (global)

```
@RestControllerAdvice
class GlobalExceptionHandler {
  @ExceptionHandler(EntityNotFoundException.class)
  ProblemDetail notFound(EntityNotFoundException e) {
    return ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND, e.getMessage());
  }
  @ExceptionHandler(MethodArgumentNotValidException.class)
  ProblemDetail invalid(MethodArgumentNotValidException e) {
    var pd = ProblemDetail.forStatus(HttpStatus.BAD_REQUEST);
    pd.setProperty("errors", e.getBindingResult().getFieldErrors().stream()
        .collect(toMap(FieldError::getField, f -> f.getDefaultMessage(), (a,b)->a)));
    return pd;
  }
  @ExceptionHandler(Exception.class)
  ProblemDetail generic(Exception e) {           // don't leak internals
    log.error("unhandled", e);
    return ProblemDetail.forStatus(HttpStatus.INTERNAL_SERVER_ERROR);
  }
}
```

Traps: consistent `ProblemDetail` (RFC 7807); never leak stack traces to clients; map domain exceptions to codes; log with correlation id.

14.5 Validation

```
public record CreateOrderReq(
    @NotNull Long customerId,
    @NotEmpty List<@Valid LineItem> items,
    @Positive BigDecimal total) {}

public record LineItem(@NotBlank String sku, @Min(1) int qty) {}
```

Traps: `@Valid` cascades into nested/collection elements; use groups for create vs update; custom `ConstraintValidator` for cross-field rules; `@Validated` on `@Service` for method-param validation.

14.6 Security (secured endpoint)

```
@PreAuthorize("hasRole('ADMIN') or #id == authentication.principal.id")
@GetMapping("/users/{id}")
UserDto get(@PathVariable Long id) { ... }
```

Traps: method + URL security; principal-based checks (owner access); never trust client-supplied identity; validate JWT (Module 6).

14.7 Transactions

```
@Transactional
public OrderDto placeOrder(CreateOrderReq req) {
  Order o = orderRepo.save(Order.from(req));
  inventory.reserve(o);                // same tx (local); external? use saga/outbox
  return OrderDto.from(o);
}
```

Traps: self-invocation bypasses proxy; checked exceptions don't roll back by default (`rollbackFor`); don't call remote APIs inside a tx (pool exhaustion); `REQUIRES_NEW` for independent units; keep transactions short.

14.8 Caching

```
@Cacheable(value="user", key="#id", sync=true)   // sync -> stampede protection
public UserDto get(Long id) { return repo.findById(id).map(UserDto::from).orElseThrow(); }

@CacheEvict(value="user", key="#dto.id())
public UserDto update(UserDto dto) { ... }
```

Traps: evict on write (not update-in-place); TTL via cache config; `sync=true` to prevent stampede; don't cache per-request/user-specific data under shared keys; `@CachePut` vs `@Cacheable` semantics.

14.9 Retry Logic

```
@Retryable(retryFor = TransientException.class,
           maxAttempts = 4,
           backoff = @Backoff(delay = 200, multiplier = 2, random = true)) // jitter
public Price fetch(String sku) { return client.get(sku); }

@Recover
Price recover(TransientException e, String sku) { return cache.last(sku); }
```

Or Resilience4j `@Retry` + `@CircuitBreaker` (Module 7). **Traps:** only retry **idempotent** ops; exponential backoff + jitter; cap attempts; combine with circuit breaker + timeout; don't retry 4xx.

14.10 Async Processing

```
@Async("taskExecutor")
public CompletableFuture<Report> generate(Long id) {
  return CompletableFuture.completedFuture(build(id));
}

@Bean
ThreadPoolTaskExecutor taskExecutor() {
  var ex = new ThreadPoolTaskExecutor();
  ex.setCorePoolSize(8); ex.setMaxPoolSize(16);
  ex.setQueueCapacity(500); ex.setThreadNamePrefix("async-");
  ex.setRejectedExecutionHandler(new ThreadPoolExecutor.CallerRunsPolicy());
  return ex;
}
```

Traps: `@Async` needs `@EnableAsync` + a proxy call (self-invocation bypass!); provide a **dedicated bounded executor** (don't use the default `SimpleAsyncTaskExecutor` which makes unbounded threads); return `CompletableFuture`/`void`; exceptions in `void` `@Async` are swallowed (use `AsyncUncaughtExceptionHandler`).

Module 14 — One-Page Cheat Sheet

Task	Do	Avoid
REST	DTOs, status codes, `ResponseEntity`	exposing entities
Pagination	cap size; keyset for deep	huge OFFSET
Upload	stream, validate type/size	load into heap
Errors	`@RestControllerAdvice` + ProblemDetail	leaking stack traces
Validation	`@Valid`, nested, groups	validating entities
Security	method + URL, owner checks	trusting client id
Tx	short, `rollbackFor`, no remote calls	self-invocation, long tx
Cache	evict on write, TTL, `sync`	shared key for user data
Retry	idempotent, backoff+jitter, cap	retrying 4xx / non-idempotent
Async	`@EnableAsync`, bounded executor	self-invoke, unbounded pool

Module 14 — Top Interview Questions

1. Design a RESTful CRUD API (status codes, DTOs, versioning).
2. Offset vs keyset pagination — when and why.
3. Safely handle large file uploads.
4. Centralized error handling with consistent schema.
5. Cascade validation into nested objects; cross-field rules.
6. Secure an endpoint for owner-or-admin access.
7. Why does `@Transactional`/`@Async`/`@Cacheable` fail on self-invocation?
8. Implement retry with backoff for a flaky dependency.
9. Cache invalidation strategy for updates.
10. Configure an async executor correctly.

Module 14 — Common Mistakes

- Exposing JPA entities directly (serialization + lazy issues).
- Self-invocation breaking proxy-based annotations.
- Unbounded async executor / retries.
- Retrying non-idempotent operations.
- Leaking internal errors to clients.

Module 14 — Mock Interview

1. *“Add pagination to a 100M-row table endpoint.”* → keyset/seek pagination + cap size; avoid deep OFFSET.
2. *“`@Async` method runs synchronously.”* → missing `@EnableAsync` or self-invocation; call through an injected bean.
3. *“Retries are hammering a failing service.”* → add backoff+jitter, cap attempts, circuit breaker; only retry idempotent/transient.
4. *“Standardize errors across the API.”* → `@RestControllerAdvice` + `ProblemDetail`, map domain exceptions, hide internals.
5. *“Upload endpoint OOMs on big files.”* → stream to object storage, don’t buffer in heap; enforce max size.

Next → Module 15: Company Interview Questions.

Module 15 — Spring Boot & Microservices Company Interviews

> A curated question bank grouped by type and by the companies that ask them. Use it as a rapid self-test: cover the answer, respond aloud, then verify against the referenced module. Big-tech Java/backend loops mix **fundamentals + system design + production debugging + behavioral**.

15.1 By Company (signature focus areas)

Company	Typical emphasis
Google	strong CS fundamentals, concurrency, complexity, design at scale, “why” behind abstractions
Amazon	Leadership Principles + scalability, idempotency, resilience, cost, ownership; bar-raiser
Microsoft	OOP/design, .NET-parallels, API design, testing, clean code
Uber	high-throughput, low-latency, geo/real-time, Kafka, sharding, consistency
Netflix	resilience (they birthed Hystrix/chaos), microservices, observability, JVM tuning
LinkedIn	Kafka (they created it), data infra, caching, feed/graph scale
VMware / Broadcom	deep Spring internals (Spring source), Tanzu, JVM, cloud-native
Oracle	JPA/Hibernate, SQL, transactions, JVM/GC internals
JPMorgan / Goldman Sachs	correctness, transactions/consistency, security, low-latency, resiliency, testing
Walmart Global Tech	microservices at retail scale, Kafka, caching, peak-traffic (Black Friday)
Adobe	API design, performance, cloud services, clean architecture
Atlassian	pragmatic system design, REST API design, multi-tenancy, testing, trade-offs

15.2 Conceptual Questions (rapid fire)

1. IoC vs DI; why constructor injection? *(M2)*
2. Full bean lifecycle; where are AOP proxies created? *(M2)*
3. How does auto-configuration work; how to override a bean? *(M3)*
4. DispatcherServlet request lifecycle. *(M4)*
5. Persistence context, dirty checking, flush vs commit. *(M5)*
6. N+1 problem — detect and fix. *(M5)*
7. `@Transactional` internals; which exceptions roll back; propagation. *(M5)*
8. Security filter chain & authentication flow. *(M6)*
9. Session vs JWT; OAuth2 vs OIDC. *(M6)*
10. Kafka delivery guarantees; ordering; consumer groups. *(M8)*
11. Caching strategies; safe invalidation. *(M9)*
12. Isolation levels & anomalies; optimistic vs pessimistic locking. *(M5, M10)*
13. G1 vs ZGC; JVM memory areas. *(M1)*
14. `CompletableFuture` composition; thread-pool sizing. *(M1)*
15. HikariCP sizing; pool exhaustion causes. *(M10, M13)*

15.3 Scenario-Based Questions

1. *"Design an idempotent payment API that tolerates client retries."* → idempotency key + unique constraint / dedup store; at-least-once safe; 409 on replay with same result. *(M7, M14)*
2. *"Order spans inventory + payment services — ensure consistency."* → orchestration saga + compensations + outbox + idempotent consumers. *(M7)*
3. *"Handle a 10x Black-Friday traffic spike."* → autoscale (HPA), cache hot data, circuit breakers + timeouts + bulkheads, async where possible, DB read replicas, load test first. *(M7, M9, M12)*
4. *"Guarantee per-user event ordering at high throughput."* → Kafka key by user → same partition. *(M8)*
5. *"Migrate a monolith to microservices."* → strangler-fig, extract bounded contexts, DB-per-service via expand/contract, anti-corruption layer. *(M7, M10)*
6. *"Run a job exactly once across N instances."* → distributed lock (Redis `SET NX PX`/Redisson) or leader election. *(M9)*

15.4 Production Issue Questions

1. Memory keeps growing until OOM — diagnose. *(M13.2)*
2. p99 latency spiked after a deploy — localize. *(M13.4)*
3. `Connection is not available` under load. *(M13.6)*
4. Kafka consumer lag growing. *(M13.7)*
5. CPU pegged at 100%. *(M13.3)*
6. Two threads hung (deadlock). *(M13.5)*
7. Cache hit ratio dropped, DB overloaded. *(M13.8)*
8. App won't start (bean/circular dependency error). *(M13.11–12)*

15.5 Architecture Questions

1. Design a URL shortener / rate limiter / notification service (apply caching, sharding, idempotency, async). *(M7–M10)*
2. How do you achieve zero-downtime deploys? *(rolling update + readiness + backward-compatible migrations — M10, M12)*
3. API gateway vs service mesh vs load balancer. *(M7)*
4. How do you secure service-to-service communication? *(mTLS / OAuth2 client-credentials / JWT resource server — M6, M7)*
5. Multi-tenancy strategies (DB-per-tenant vs shared schema + tenant id). *(M10)*
6. How do you make a service observable? *(metrics+logs+traces, RED/USE, SLOs — M11)*

15.6 Debugging Questions

1. Which tools capture thread state, heap, GC, CPU? *(jstack, jmap/MAT, GC logs, async-profiler/JFR, Actuator — M13)*
2. What do you capture before restarting a sick JVM? *(thread + heap dump, GC log.)*
3. How do you find which service causes a latency spike? *(distributed trace — M11)*
4. How do you detect a Java deadlock vs thread starvation? *(jstack deadlock section vs pool saturation/blocked-on-I/O — M13)*

15.7 Coding Questions

1. RESTful CRUD with pagination, validation, error handling. *(M14)*
2. Streaming file upload with validation. *(M14)*
3. Retry with exponential backoff + jitter and fallback. *(M14, M7)*
4. `@Cacheable` with stampede protection and eviction on write. *(M14, M9)*
5. Core Java: group/aggregate with Streams; sort by multiple fields; thread-safe counter; produce/merge parallel `CompletableFuture`s. *(M1)*
6. Implement a token-bucket rate limiter (in-memory or Redis). *(M9)*

15.8 Follow-up Questions (interviewers dig deeper)

- "You said constructor injection — how does Spring know which constructor?" *(single constructor auto; else `@Autowired`.)*
- "You used JWT — how do you revoke one?" *(short TTL + refresh rotation + denylist.)*
- "You added an index — why might the planner ignore it?" *(low selectivity, stale stats, non-SARGable predicate, small table.)*
- "Retry — what if the operation isn't idempotent?" *(don't retry, or add an idempotency key.)*
- "Circuit breaker OPEN — what does the user see?" *(fast failure + fallback.)*
- "You cached it — how do you keep it consistent?" *(evict on write, TTL, accept eventual consistency, or write-through.)*
- "Saga failed midway — how do you recover?" *(compensating transactions; idempotent + retriable steps; monitor stuck sagas.)*

15.9 Behavioral / System-design framing (senior signal)

- **STAR** for behavioral (Situation, Task, Action, Result); quantify impact.
- Amazon: map stories to **Leadership Principles** (Ownership, Bias for Action, Dive Deep, Deliver Results).
- For design: **clarify requirements → estimate scale (QPS, data) → API → data model → components → bottlenecks → trade-offs → failure modes**. State assumptions and trade-offs explicitly; there is no single right answer.
- Always end an answer with trade-offs and how you'd validate/monitor it.

Module 15 — Final Prep Cheat Sheet

Bucket	Nail these
Spring internals	bean lifecycle, proxies/self-invocation, auto-config, DispatcherServlet
Data	persistence context, N+1, @Transactional propagation/rollback, isolation, locking
Security	filter chain, AuthN flow, JWT vs session, OAuth2/OIDC, CSRF/CORS
Distributed	saga, outbox, idempotency, circuit breaker/retry/timeout/bulkhead
Messaging	partitions/ordering, consumer groups, delivery guarantees, DLQ
Caching	strategies, invalidation, stampede, distributed lock
JVM	memory areas, GC (G1/ZGC), thread pools, CompletableFuture
Ops	HikariCP, probes, observability (metrics/logs/traces), K8s objects
Debugging	symptom → evidence → isolate → fix → prevent; jstack/jmap/traces

Module 15 — Mock Interview (mixed loop)

1. *(Concept)* “Why does `@Transactional` on a private method do nothing?” → proxy-based AOP only advises public methods called through the proxy.
2. *(Scenario)* “Design an idempotent ‘create payment’ endpoint.” → idempotency key + unique constraint; return prior result on replay.
3. *(Production)* “Latency spiked after deploy; DB CPU normal.” → trace shows a downstream call without timeout; add timeout + circuit breaker; consider the new code path/N+1.
4. *(Architecture)* “Zero-downtime schema change on a huge table.” → expand/contract, backward-compatible, batched backfill, rolling deploy.
5. *(Coding)* “Rate-limit 100 req/min/user across instances.” → Redis token bucket (atomic), 429 + `Retry-After`.
6. *(Follow-up)* “How would you prove your fix worked?” → metrics/SLO dashboards before/after, load test, add an alert + regression test.

You’ve completed all 15 modules.

Review the **Master PDF** for the full curriculum, then drill the per-module cheat sheets and mock interviews until you can answer each aloud without notes. Good luck — you’re interview-ready.